ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN



BÁO CÁO

Học phần: Phân tích dữ liệu

BÀI TẬP

Phân tích dữ liệu Chương 13

Giảng viên hướng dẫn: ThS. Đỗ Như Tài

Thành viên nhóm:

Nguyễn Vũ Hào – MSSV: 3122410098

Trần Duy Khương – MSSV: 3122410192

Võ Thị Thương – MSSV: 3122410408

Tp.Hồ Chí Minh, 2025

Họ và Tên	**Mã số sinh viên**	**Phân công công việc**
Nguyễn Vũ Hào	3122410098	- Làm mục 13.4 và 13.5 - Viết doc chương 4, 5, 6 - Làm slide chương 5
Trần Duy Khương	3122410192	- Làm mục 13.3 - Viết doc chương 3 - Làm slide chương 3, 4
Võ Thị Thương	3122410408	- Làm mục 13.1 và 13.2 - Viết doc chương 1, 2 - Làm slide chương 1, 2

MỤC LỤC

MỤC LỤC ẢNH 4

LỜI CẢM ƠN 8

LỜI MỞ ĐẦU 9

Chương 1: Giới thiệu 10

1. Bối cảnh và tầm quan trọng của Phân tích dữ liệu 10

2. Mục tiêu 10

Chương 2: Phân tích dữ liệu Bitly từ 1.USA.gov 12

1. Tổng quan về bộ dữ liệu Bitly (Dữ liệu JSON) 12

2. Phân tích múi giờ (Time Zones) 12

2.1 Sử dụng Dictionary Thuần túy (Python Native) 13

2.2 Sử dụng collections.defaultdict 14

2.3 Sử dụng collection.Counter 14

2.4 Sử dụng Thư viện pandas 15

3. Phân tích hành vi của người dùng theo hệ điều hành 18

Chương 3: Phân tích Bộ dữ liệu MovieLens 1M 24

1. Tổng quan về bộ dữ liệu 24

2. Hợp nhất dữ liệu (Merging Data) 27

3. Xếp hạng Trung bình theo Tiêu đề phim và Giới tính 28

4. Đo lường Sự khác biệt về Xếp hạng 29

5. Phân tích Xếp hạng theo Thể loại (Genre) và Độ tuổi 32

Chương 4: Xu hướng Tên Trẻ Sơ Sinh ở Mỹ 1880-2010 35

1. Tổng quan và chuẩn bị dữ liệu 35

2. Phân tích Xu hướng Đặt tên cụ thể 39

3. Đo lường sự đa dạng tên 41

4. Cuộc cách mạng "Chữ cái Cuối cùng" 43

5. Xu hướng Đổi Giới tính của Tên 46

Chương 5: Phân tích Cơ sở dữ liệu thực phẩm USDA 48

1. Tổng quan và chuẩn bị dữ liệu 48

2. Xử lý dữ liệu 49

3. Phân tích và trực quan hóa dữ liệu 52

Chương 6: Phân tích Cơ sở dữ liệu Ủy ban Bầu cử Liên bang năm 2012 55

1. Tải Dữ Liệu Và Kiểm Tra Thông Tin Cơ Bản 55

2. Truy suất một bảng ghi mẫu 56

3. Lấy danh sách Ứng cử viên duy nhất 56

4. Ánh xạ Đáng phái cho Ứng cử viên 57

5. Lọc dữ liệu đóng góp dương 58

6. Thống kê nghề nghiệp 61

7. Làm Sạch Dữ Liệu Nghề Nghiệp Và Nhà Tuyển Dụng 61

8. Pivot Table Theo Nghề Nghiệp Và Đảng 63

9. Vẽ Biểu Đồ Theo Nghề Nghiệp 65

10. Phân Loại Đóng Góp Theo Khoảng (Bucketing) 66

11. Nhóm Và Tính Toán Theo Khoảng Đóng Góp 68

12. Thống kê theo Tiểu bang 71

13. Tính Tỷ Lệ Phần Trăm Theo Tiểu Bang 72

Chương 7: Kết luận 74

1. Tổng hợp các kỹ thuật nổi bật trong 5 phần phân tích 74

2. So sánh hiệu quả giữa các phương pháp 75

3. Bài học kinh nghiệm khi làm việc với dữ liệu thực tế 76

4. Hướng phát triển và ứng dụng trong tương lai 77

MỤC LỤC ẢNH

Hình 1. 1 Dữ liệu ban đầu của file example.txt 12
Hình 1. 2 Dữ liệu sau khi chuyển sang từ điển (dictionary) trong Python 12
Hình 1. 3 Dữ liệu khi trích xuất có lỗi KeyError 13
Hình 1. 4 Dữ liệu sau khi xử lý lỗi KeyError 13
Hình 1. 5 Tần suất của từng TimeZones theo Dictionary 14
Hình 1. 6 Tần suất của từng TimeZones theo Collection 14
Hình 1. 7 Tần suất của từng TimeZones theo Counter() 15
Hình 1. 8 Những thông tin cơ bản của tập Dataframe 16
Hình 1. 9 Tần suất của TimeZones dựa trên pandas 16
Hình 1. 10 Dữ liệu sau khi được làm sạch 17
Hình 1. 11 Biểu đồ top 10 tần suất TimeZones hàng đầu của dữ liệu 18
Hình 1. 12 Một số token phổ biến trong dữ liệu 19
Hình 1. 13 Phân loại hệ điều hành trong bộ dữ liệu 19
Hình 1. 14 Dữ liệu sau khi được nhóm theo tz và os 20
Hình 1. 15 Tổng số bản ghi của từng TimeZones 20
Hình 1. 16 Bảng dữ liệu sau khi cắt bỏ 10 chỉ số cuối cùng (múi giờ lớn nhất) . 21
Hình 1. 17 Sử dụng take để chọn các hàng theo thứ tự (múi giờ lớn nhất) 21
Hình 1. 18 Biểu đồ số lượng người dùng Windows và Non-Windows trong từng múi giờ. 22
Hình 1. 19 Biểu đồ trực quan hóa tỷ lệ phần trăm người dùng Windows và Non-Windows 23
Hình 2. 1 Bộ dữ liệu ban đầu của users.dat 24
Hình 2. 2 Bộ dữ liệu ban đầu của movies.dat 25
Hình 2. 3 Bộ dữ liệu ban đầu của ratings.dat 25
Hình 2. 4 Bảng chứa dữ liệu users(5 dòng đầu) 26
Hình 2. 5 Bảng chứa dữ liệu ratings (5 dòng đầu) 26
Hình 2. 6 Bảng chứa dữ liệu movies (5 dòng đầu) 27
Hình 2. 7 Bảng dữ liệu sau khi đã gộp 3 bảng với nhau 27

Hình 2. 8 Điểm đánh giá trung bình của mỗi bộ phim theo giới tính 28
Hình 2. 9 Các bộ phim nhận được ít nhất 250 lượt đánh giá 29
Hình 2. 10 Lọc các hàng tương ứng từ DataFrame của các bộ phim 29
Hình 2. 11 Các bộ phim được nữ khán giả đánh giá cao nhất 30
Hình 2. 12 Các phim được Phụ nữ ưa thích hơn xếp theo cột diff 30
Hình 2. 13 Các phim được Đàn ông ưa thích xếp theo cột diff 31
Hình 2. 14 Tính độ lệch chuẩn sự phân tán xung quanh 31
Hình 2. 15 Phim có sự khác biệt về đánh giá lớn 32
Hình 2. 16 Bộ dữ liệu cột genre ban đầu 32
Hình 2. 17 Tách dữ liệu của cột genres sử dụng split() 33
Hình 2. 18 Bảng dữ liệu cho từng thể loại từng bộ phim 33
Hình 2. 19 Dữ liệu sau khi đã hợp nhất 34
Hình 2. 20 Xếp hạng trung bình của từng thể loại phim giữa các nhóm tuổi khác nhau 34
Hình 3. 1 Bộ dữ liệu ban đầu của Yob1880.txt 35
Hình 3. 2 Dữ liệu sau khi được đọc và thêm vào bảng 36
Hình 3. 3 Tổng số ca sinh theo giới tính và năm 36
Hình 3. 4 Dữ liệu sau khi gộp và thêm cột year 37
Hình 3. 5 Tổng hợp dữ liệu ở cấp độ year và sex 37
Hình 3. 6 Biểu đồ tổng số ca sinh theo giới tính và năm 38
Hình 3. 7 Bảng dữ liệu phần trăm trẻ sơ sinh được đặt tên cụ thể 38
Hình 3. 8 Xác minh dữ liệu đúng đắn 39
Hình 3. 9 Bảng dữ liệu sau khi lọc ra 1000 tên đặt nhiều nhất 39
Hình 3. 10 Biểu đồ của các tên truyền thống theo từng năm 40
Hình 3. 11 Biểu đồ của các tên truyền thống theo từng năm(2) 40
Hình 3. 12 Biểu đồ tỷ lệ sinh của tổng 1000 tên hàng đầu 41
Hình 3. 13 Bảng các tên con trai từ năm 2010 42
Hình 3. 14 Số lượng Tên cần thiết để Chiếm 50% Tổng số Ca sinh theo các năm 42
Hình 3. 15 Biểu đồ sự da dạng tên của trẻ em qua từng năm 43
Hình 3. 16 Bảng dữ liệu về sự phân bố tên trẻ em theo chữ cái cuối 43

Hình 3. 17 Tỷ lệ tổng số ca sinh cho mỗi giới tính 44
Hình 3. 18 Biểu đồ sự phân hóa tên bé trai kết thúc bằng chữ cái 44
Hình 3. 19 Biểu đồ sự phân hóa tên bé gái kết thúc bằng chữ cái 45
Hình 3. 20 Biểu đồ thể hiện xu hướng các chữ cái cuối phổ biến 46
Hình 3. 21 Tập dữ liệu cho các tên bắt đầu bằng "Lesl" 46
Hình 3. 22 Sự chuyển đổi trong mức độ ưa chuộng giới tính 47
Hình 3. 23 Biểu đồ phân tích sự phân chia giới tính theo nhóm tên “Lesl” 47
Hình 4. 1 Thông tin thành phần dinh dưỡng đầu tiên của thực phẩm đầu tiên ... 48
Hình 4. 2 Bảng danh sách thành phần dinh dưỡng của thực phẩm đầu tiên 49
Hình 4. 3 Bảng thông tin của DataFrame info 49
Hình 4. 4 Thông tin của DataFrame info 50
Hình 4. 5 Kết quả của 10 giá trị đầu tiên về sự phân bổ của các nhóm thực phẩm 50
Hình 4. 6 Thông tin DataFrame nutrients sau khi khởi tạo 51
Hình 4. 7 Thông tin của info sau khi đổi tên cột 51
Hình 4. 8 Thông tin của nutrients sau khi đổi tên cột 52
Hình 4. 9 DataFrame tổng hợp sau khi merge info và nutrients 52
Hình 4. 10 Thông tin của dữ liệu thứ 30001 52
Hình 4. 11 Biểu đồ Thanh Ngang so sánh hàm lượng Kẽm (Zinc, Zn) Trung vị giữa các nhóm thực phẩm 53
Hình 4. 12 Kết quả thống kê các giá trị cực đại của nhóm dinh dưỡng "Axit amin" 54
Hình 5. 1 Thông tin tổng quát về DataFrame 55
Hình 5. 2 Bảng giải thích dữ liệu 56
Hình 5. 3 Dữ liệu dòng thứ 123456 dưới dạng Series 56
Hình 5. 4 Mảng danh sách ứng viên 57
Hình 5. 5 5 dòng đầu tiên DataFrame sau khi đã xử lý 57
Hình 5. 6 Thông tin DataFrame sau khi thêm cột mới 57
Hình 5. 7 Số lượng giá trị trong cột party 58
Hình 5. 8 Biểu đồ histogram cho cột 'party' 58
Hình 5. 9 Kết quả kiểm tra contributions âm/dương 59

Hình 5. 10 Kết quả kiểm tra số lượng giá trị thiếu ở mỗi cột 59
Hình 5. 11 Thông tin DataFrame sau khi lọc 60
Hình 5. 12 Thông tin DataFrame mới chỉ chứa các dòng là ứng viên Obama hoặc Romney 60
Hình 5. 13 5 dòng đầu tiên của DataFrame đã lọc 60
Hình 5. 14 Top 10 nghề nghiệp phổ biến nhất 61
Hình 5. 15 Tần suất nghề nghiệp sau khi làm sạch 62
Hình 5. 16 Tần suất nhà tuyển dụng trước khi làm sạch 62
Hình 5. 17 Tần suất nhà tuyển dụng sau khi làm sạch 62
Hình 5. 18 5 dòng đầu tiên của DataFrame sau khi làm sạch 62
Hình 5. 19 Thông tin của pivot table 63
Hình 5. 20 5 dòng đầu tiên của pivot table 63
Hình 5. 21 Các hàng có tổng tiền (theo cột) > 2 triệu 64
Hình 5. 22 Biểu đồ ngang từ DataFrame over_2mm 65
Hình 5. 23 Label phân loại số tiền vào bins 66
Hình 5. 24 Bảng thống kê số lượng người quyên góp trong mỗi khoảng tiền 67
Hình 5. 25 Bảng tổng số tiền theo từng khoảng và đảng phái 67
Hình 5. 26 Biểu đồ thanh từ thống kê lượng người quyên góp trong mỗi khoảng tiền của mỗi đảng 68
Hình 5. 27 Bảng đã được unstack ra 69
Hình 5. 28 Bảng tổng tiền theo nhóm 69
Hình 5. 29 Bảng tỷ lệ phần trăm theo hàng 70
Hình 5. 30 Biểu đồ ngang biểu diễn tỷ lệ phần trăm theo hàng (loại bỏ 2 dòng cuối) 70
Hình 5. 31 Bảng tổng tiền theo tiểu bang và ứng cử viên 71
Hình 5. 32 Bảng danh sách tiểu bang có tổng > 100.000 (10 dòng đầu tiên) 71
Hình 5. 33 Bảng Tỷ Lệ Phần Trăm Theo Tiểu Bang sắp xếp theo cột Obama 72
Hình 5. 34 Bảng danh sách top 10 bang có tỷ lệ đóng góp cao nhất cho Romney 73

LỜI CẢM ƠN

Chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ThS. Đỗ Như Tài, Giảng viên hướng dẫn học phần Phân tích dữ liệu tại Trường Đại học Sài Gòn. Thầy đã tận tình hướng dẫn, truyền đạt những kiến thức quý báu và hỗ trợ chúng em trong suốt quá trình học tập và thực hiện báo cáo này.

Những kiến thức và kinh nghiệm mà thầy chia sẻ, đặc biệt là về Thống kê Mô tả, Trực quan hóa Dữ liệu, và các phương pháp Phân tích Đơn biến/Hai biến, không chỉ giúp nhóm sinh viên chúng em hiểu rõ hơn về các phương pháp phân tích dữ liệu mà còn rèn luyện khả năng xử lý và trình bày dữ liệu một cách khoa học.

Chúng em cũng xin chân thành cảm ơn Quý Thầy/Cô trong Khoa đã tạo điều kiện thuận lợi trong suốt quá trình học tập. Đồng thời, chúng em xin gửi lời cảm ơn đến các bạn cùng lớp đã chia sẻ, trao đổi, và hỗ trợ trong quá trình thực hiện đề tài.

Mặc dù nhóm đã nỗ lực cố gắng hoàn thiện báo cáo một cách tốt nhất, song do kiến thức và kinh nghiệm còn hạn chế, bài báo cáo chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những góp ý quý báu từ Thầy để nhóm có thể hoàn thiện hơn trong tương lai.

Chúng em xin chân thành cảm ơn!

LỜI MỞ ĐẦU

Báo cáo này được biên soạn từ Chương 13 của tài liệu phân tích dữ liệu, nhằm giới thiệu và minh họa việc ứng dụng các kỹ thuật đã được trình bày trong cuốn sách này vào các bộ dữ liệu thực tế. Mục tiêu chính là chỉ ra cách sử dụng các công cụ mạnh mẽ như pandas và NumPy để trích xuất ý nghĩa từ dữ liệu thô.

Mỗi phần phân tích trong báo cáo sẽ tập trung vào một bộ dữ liệu cụ thể, bao gồm:

Bitly Data từ 1.USA.gov: Phân tích dữ liệu URL rút gọn để khám phá hành vi người dùng và địa lý.

MovieLens 1M Dataset: Phân tích xếp hạng phim để tìm hiểu về sở thích và sự khác biệt giữa các nhóm người dùng.

US Baby Names 1880–2010: Phân tích xu hướng đặt tên trẻ sơ sinh để đo lường sự thay đổi xã hội và văn hóa qua hơn một thế kỷ.

Các kỹ thuật được trình bày trong báo cáo này có thể được áp dụng rộng rãi cho nhiều loại bộ dữ liệu khác, giúp người đọc rèn luyện và làm quen với các công cụ phân tích dữ liệu hiện đại. Tất cả các bộ dữ liệu ví dụ được sử dụng đều có thể tìm thấy trong kho lưu trữ GitHub đi kèm tài liệu.

Chương 1: Giới thiệu

1. Bối cảnh và tầm quan trọng của Phân tích dữ liệu

Trong kỷ nguyên kỹ thuật số, dữ liệu được xem là tài sản chiến lược hàng đầu. Tuy nhiên, bản thân dữ liệu thô thường rời rạc và không có cấu trúc. Khả năng chuyển đổi dữ liệu thô thành kiến thức có ý nghĩa, có thể hành động (actionable insights) là cốt lõi của Khoa học Dữ liệu và là yếu tố then chốt thúc đẩy các quyết định kinh doanh, nghiên cứu khoa học và chính sách công.

Báo cáo này là một nghiên cứu thực nghiệm nhằm minh họa việc áp dụng các công cụ và kỹ thuật xử lý dữ liệu hiện đại, đặc biệt là thông qua các thư viện lập trình Python mạnh mẽ như pandas và NumPy. Mục tiêu chính là làm sáng tỏ quá trình khai thác ý nghĩa từ các bộ dữ liệu trong thế giới thực thông qua các ví dụ thực tế đa dạng. Các kỹ thuật được trình bày ở đây không chỉ giới hạn trong các trường hợp cụ thể này mà còn có thể được áp dụng rộng rãi cho tất cả các loại bộ dữ liệu khác trong nhiều lĩnh vực.

2. Mục tiêu

Mục tiêu chính của báo cáo này là minh họa chi tiết cách thức thực hiện các phân tích phức tạp, từ việc chuẩn bị dữ liệu thô cho đến việc đưa ra các kết quả thống kê có giá trị, thông qua việc tập trung vào năm bộ dữ liệu lớn và đa dạng như sau:

Phân tích Dữ liệu Bitly từ 1.USA.gov: Trình bày quy trình xử lý dữ liệu web ẩn danh được định dạng JSON, bao gồm việc trích xuất và thống kê dữ liệu địa lý như múi giờ (tz), và phân tích chuỗi tác nhân (a) để phân loại người dùng (ví dụ: xác định tỷ lệ người dùng sử dụng hệ điều hành Windows so với Non-Windows).

Phân tích Bộ dữ liệu MovieLens 1M: Minh họa kỹ thuật hợp nhất (merging) các bảng dữ liệu riêng biệt (xếp hạng, thông tin người dùng, thông tin phim) để thực hiện các phân tích thống kê đa chiều. Cụ thể, báo cáo sẽ tính toán xếp hạng trung bình theo các nhóm giới tính và độ tuổi, đồng thời đo

lường mức độ bất đồng (độ lệch chuẩn) trong xếp hạng giữa các nhóm người xem khác nhau.

Phân tích Tên Trẻ em Hoa Kỳ 1880–2010: Hợp nhất dữ liệu tên qua nhiều thập kỷ để phân tích xu hướng đặt tên theo thời gian, đo lường sự đa dạng hóa tên gọi, và khám phá các hiện tượng văn hóa đặt tên, chẳng hạn như "cuộc cách mạng chữ cái cuối cùng" trong việc lựa chọn tên.

Phân tích Cơ sở dữ liệu Thực phẩm USDA: Tập trung vào việc xử lý dữ liệu dinh dưỡng phức tạp ở định dạng JSON, chuyển đổi chúng một cách có hệ thống thành các cấu trúc bảng (DataFrame) để dễ dàng thao tác, và hợp nhất thông tin để tiến hành phân tích hàm lượng dinh dưỡng theo nhóm thực phẩm.

Phân tích Cơ sở dữ liệu Ủy ban Bầu cử Liên bang (FEC) 2012: Phân tích các khoản đóng góp chiến dịch bằng cách phân loại các khoản đóng góp theo đảng phái chính trị (Obama so với Romney), thực hiện thống kê chi tiết theo nghề nghiệp, chủ lao động của người đóng góp, và phân nhóm số tiền quyên góp để hiểu rõ hơn về nguồn tài trợ chiến dịch.

Chương 2: Phân tích dữ liệu Bitly từ 1.USA.gov

1. Tổng quan về bộ dữ liệu Bitly (Dữ liệu JSON)

Nghiên cứu này bắt đầu bằng việc phân tích bộ dữ liệu Bitly thu thập từ các liên kết rút gọn của Chính phủ Hoa Kỳ. Vào năm 2011, dịch vụ rút gọn URL Bitly đã hợp tác với trang web USA.gov để cung cấp một nguồn cấp dữ liệu ẩn danh, thu thập thông tin từ những người dùng rút gọn các liên kết có đuôi .gov hoặc .mil. Trong năm đó, luồng dữ liệu trực tiếp (live feed) cùng với các bản chụp dữ liệu theo giờ (hourly snapshots) đều được công bố công khai dưới dạng tệp văn bản có thể tải xuống. Mặc dù dịch vụ này đã ngừng hoạt động (tính đến năm 2022), báo cáo này sử dụng một tệp dữ liệu đã được bảo tồn (example.txt) để minh họa các kỹ thuật phân tích dữ liệu web thực tế.

{ "a": "Mozilla\/5.0 (Windows NT 6.1; WOW64) AppleWebKit\/535.11 (KHTML, like Gecko) Chrome\/17.0.963.78 Safari\/535.11", "c": "US", "nk": 1, "tz": "America\/New_York", "gr": "MA", "g": "A6qOVH", "h": "wfLQtf", "l": "orofrog", "al": "en-US,en;q=0.8", "hh": "1.usa.gov", "r": "http:\/\/www.facebook.com\/l\/7AQEFzjSi\/1.usa.gov\/wfLQtf", "u": "http:\/\/www.ncbi.nlm.nih.gov\/pubmed\/22415991", "t": 1331923247, "hc": 1331822918, "cy": "Danvers", "ll": [42.576698, -70.954903] }
{ "a": "GoogleMaps\/RochesterNY", "c": "US", "nk": 0, "tz": "America\/Denver", "gr": "UT", "g": "mwszk8", "h": "mwszk8", "l": "bitly", "hh": "j.mp", "r": "http:\/\/www.AwareMap.com\/", "u": "http:\/\/www.monroecounty.gov\/etc\/911\/rss.php", "t": 1331923249, "hc": 1308262393, "cy": "Provo", "ll": [40.218102, -111.613297] }
{ "a": "Mozilla\/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64; Trident\/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; Media Center PC 6.0; .NET4.0C; .NET4.0E; InfoPath.3)", "c": "US", "nk": 1, "tz": "America\/New_York", "gr": "DC", "g": "xxr3Qb", "h": "xxr3Qb", "l": "bitly", "al": "en-US", "hh": "1.usa.gov", "r": "http:\/\/t.co\/03elZC4Q", "u": "http:\/\/boxer.senate.gov\/en\/press\/releases\/031612.cfm", "t": 1331923250, "hc": 1331919941, "cy": "Washington", "ll": [38.900700, -77.043098] }
{ "a": "Mozilla\/5.0 (Macintosh; Intel Mac OS X 10_6_8) AppleWebKit\/534.52.7 (KHTML, like Gecko) Version\/5.1.2 Safari\/534.52.7", "c": "BR", "nk": 0, "tz": "America\/Sao_Paulo", "gr": "27", "g": "zCaLwp", "h": "zUtuOu", "l": "alelex88", "al": "pt-br", "hh": "1.usa.gov", "r": "direct", "u": "http:\/\/apod.nasa.gov\/apod\/ap120312.html", "t": 1331923249, "hc": 1331923068, "cy": "Braz", "ll": [-23.549999, -46.616699] }
{ "a": "Mozilla\/5.0 (Windows NT 6.1; WOW64) AppleWebKit\/535.11 (KHTML, like Gecko) Chrome\/17.0.963.79 Safari\/535.11", "c": "US", "nk": 0, "tz": "America\/New_York", "gr": "MA", "g": "9b6kNl", "h": "9b6kNl", "l": "bitly", "al": "en-US,en;q=0.8", "hh": "bit.ly", "r": "http:\/\/www.shrewsbury-ma.gov\/selco\/", "u": "http:\/\/www.shrewsbury-ma.gov\/egov\/gallery\/1341273686672998.png", "t": 1331923251, "hc": 1273672411, "cy": "Shrewsbury", "ll": [42.286499, -71.714699] }
{ "a": "Mozilla\/5.0 (Windows NT 6.1; WOW64) AppleWebKit\/535.11 (KHTML, like Gecko) Chrome\/17.0.963.79 Safari\/535.11", "c": "US", "nk": 0, "tz": "America\/New_York", "gr": "MA", "g": "axNK8c", "h": "axNK8c", "l": "bitly", "al": "en-US,en;q=0.8", "hh": "bit.ly", "r": "http:\/\/www.shrewsbury-ma.gov\/selco\/", "u": "http:\/\/www.shrewsbury-ma.gov\/egov\/gallery\/1341273686675393.png", "t": 1331923252, "hc": 1273672506, "cy": "Shrewsbury", "ll": [42.286499, -71.714699] }
{ "a": "Mozilla\/5.0 (Windows NT 5.1) AppleWebKit\/535.11 (KHTML, like Gecko) Chrome\/17.0.963.79 Safari\/535.11", "c": "PL", "nk": 0, "tz": "Europe\/Warsaw", "gr": "77", "g": "wcndER", "h": "zkpJBR", "l": "bnjacobs", "al": "pl-PL,pl;q=0.8,en-US;q=0.6,en;q=0.4", "hh": "1.usa.gov", "r": "http:\/\/plus.url.google.com\/url?sa=z&n=1331923253510&url=http%3A%2F%2F1.usa.gov%2FzkpJBR&usg=TF5iGSqKl_GSADtHqTqjuU2yMIY.", "u": "http:\/\/www.nasa.gov\/mission_pages\/nustar\/main\/index.html", "t": 1331923255, "hc": 1331922854, "cy": "Luban", "ll": [51.116699, 15.283300] }
{ "a": "Mozilla\/5.0 (Windows NT 6.1; rv:2.0.1) Gecko\/20100101 Firefox\/4.0.1", "c": null, "nk": 0, "tz": "", "g": "wcndER", "h": "zkpJBR", "l": "bnjacobs", "al": "bg,en-us;q=0.7,en;q=0.3", "hh": "1.usa.gov", "r": "http:\/\/www.facebook.com\/", "u": "http:\/\/www.nasa.gov\/mission_pages\/nustar\/main\/index.html", "t": 1331923255, "hc": 1331922854 }

Hình 1. 1 Dữ liệu ban đầu của file example.txt

Mỗi dòng trong tệp dữ liệu này chứa một dạng dữ liệu web phổ biến gọi là JSON (JavaScript Object Notation). JSON là một định dạng văn bản nhẹ, độc lập với ngôn ngữ, được sử dụng để trao đổi dữ liệu. Dữ liệu sau khi đọc vào Python sẽ được chuyển đổi thành một danh sách các từ điển (dictionary), trong đó mỗi từ điển đại diện cho một bản ghi truy cập.


```
{'a': 'Mozilla/5.0 (Windows NT 6.1; WOW64) AppleWebKit/535.11 (KHTML, like Gecko) Chrome/17.0.963.78 Safari/535.11',
 'c': 'US',
 'nk': 1,
 'tz': 'America/New_York',
 'gr': 'MA',
 'g': 'A6qOVH',
 'h': 'wfLQtf',
 'l': 'orofrog',
 'al': 'en-US,en;q=0.8',
 'hh': '1.usa.gov',
 'r': 'http://www.facebook.com/l/7AQEFzjSi/1.usa.gov/wfLQtf',
 'u': 'http://www.ncbi.nlm.nih.gov/pubmed/22415991',
 't': 1331923247,
 'hc': 1331822918,
 'cy': 'Danvers',
 'll': [42.576698, -70.954903]}
```


Hình 1. 2 Dữ liệu sau khi chuyển sang từ điển (dictionary) trong Python

2. Phân tích múi giờ (Time Zones)

Quá trình trích xuất múi giờ ban đầu gặp phải thách thức về tính toàn vẹn của dữ liệu. Do không phải tất cả các bản ghi JSON đều chứa trường tz, việc cố gắng truy cập trường này một cách trực tiếp sẽ gây ra lỗi KeyError.

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[23], line 1
----> 1 time_zones = [rec["tz"] for rec in records]

KeyError: 'tz'
```

Hình 1. 3 Dữ liệu khi trích xuất có lỗi KeyError

Việc xử lý dữ liệu thiếu này được thực hiện bằng cách thêm điều kiện kiểm tra if "tz" in rec trong quá trình trích xuất, tạo ra danh sách các múi giờ bao gồm cả các giá trị không xác định, được biểu thị bằng chuỗi trống (").

```
time_zones = [rec["tz"] for rec in records if "tz" in rec]
time_zones[:10]
```

```
['America/New_York',
 'America/Denver',
 'America/New_York',
 'America/Sao_Paulo',
 'America/New_York',
 'America/New_York',
 'Europe/Warsaw',
 '',
 '',
 '']
```

Hình 1. 4 Dữ liệu sau khi xử lý lỗi KeyError

Mặc dù việc lọc này đã loại bỏ các bản ghi không hoàn chỉnh, danh sách múi giờ thu được vẫn chứa một lượng đáng kể chuỗi trống ("). Những chuỗi này đại diện cho các trường hợp mà trường tz tồn tại trong bản ghi JSON, nhưng giá trị của nó lại bị thiếu hoặc không thể xác định được từ dịch vụ Bitly. Do đó, chuỗi trống cần được xử lý riêng biệt với các giá trị bị thiếu hoàn toàn (sẽ được biểu thị là NaN trong pandas).

Việc đếm tần suất xuất hiện của các múi giờ sau khi đã được trích xuất vào danh sách time_zones có thể được thực hiện bằng ba phương pháp chính, từ thủ công bằng Python thuần túy đến việc sử dụng các công cụ chuyên biệt và thư viện khoa học dữ liệu.

2.1 Sử dụng Dictionary Thuần túy (Python Native)

Đây là phương pháp lập trình thủ công cơ bản, sử dụng cấu trúc dictionary để theo dõi số lần xuất hiện. Một dictionary trống được khởi tạo, sau đó chương trình sẽ lặp qua danh sách múi giờ đã trích xuất. Đối với mỗi múi giờ, nếu nó đã tồn tại trong dictionary đếm, giá trị tương ứng sẽ được tăng lên (increment); nếu không, nó sẽ được gán giá trị khởi tạo là 1. Phương pháp này minh họa nguyên lý đếm tần suất cốt lõi.

```
def get_counts(sequence):
    counts = {}
    for x in sequence:
        if x in counts:
            counts[x] += 1
        else:
            counts[x] = 1
    return counts
```

Hình 1. 5 Tần suất của từng TimeZones theo Dictionary

2.2 Sử dụng collections.defaultdict

Cấu trúc này, nằm trong mô-đun collections, giúp đơn giản hóa quá trình đếm thủ công bằng cách loại bỏ nhu cầu kiểm tra điều kiện tồn tại của khóa. Khi khởi tạo với defaultdict(int), bất kỳ khóa nào được truy cập lần đầu sẽ tự động được gán giá trị mặc định là 0, cho phép tăng giá trị trực tiếp mà không cần kiểm tra if/else.

```
from collections import defaultdict

def get_counts2(sequence):
    counts = defaultdict(int) # values will initialize to 0
    for x in sequence:
        counts[x] += 1
    return counts
```

Hình 1. 6 Tần suất của từng TimeZones theo Collection

2.3 Sử dụng collection.Counter

Đây là phương pháp đơn giản nhất và được khuyến nghị nhất trong thư viện chuẩn. Lớp Counter được thiết kế chuyên biệt để thực hiện việc đếm tần suất xuất hiện của các phần tử trong một chuỗi lặp

(iterable). Bằng cách gọi Counter(time_zones), toàn bộ quá trình đếm được thực hiện một cách tự động và tối ưu.

```
from collections import Counter
counts = Counter(time_zones)
counts.most_common(10)
```

```
[('America/New_York', 1251),
 ('', 521),
 ('America/Chicago', 400),
 ('America/Los_Angeles', 382),
 ('America/Denver', 191),
 ('Europe/London', 74),
 ('Asia/Tokyo', 37),
 ('Pacific/Honolulu', 36),
 ('Europe/Madrid', 35),
 ('America/Sao_Paulo', 33)]
```

Hình 1. 7 Tần suất của từng TimeZones theo Counter()

Kết quả sử dụng Counter cho thấy 10 múi giờ phổ biến nhất, trong đó America/New_York chiếm ưu thế với 1251 lượt và chuỗi trống '' (đại diện cho múi giờ không xác định/bị thiếu) là giá trị đứng thứ hai với 521 lượt.

2.4 Sử dụng Thư viện pandas

Việc chuyển sang sử dụng thư viện pandas không chỉ giúp việc phân tích trở nên trực quan mà còn hiệu quả hơn nhiều, đặc biệt khi xử lý các bộ dữ liệu lớn.

Bước đầu tiên là chuyển đổi danh sách các bản ghi JSON (records) đã được load thành cấu trúc dữ liệu pd.DataFrame. Cấu trúc DataFrame này cho phép xử lý dữ liệu dạng bảng với hiệu suất cao.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3560 entries, 0 to 3559
Data columns (total 18 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   a            3440 non-null   object
 1   c            2919 non-null   object
 2   nk           3440 non-null   float64
 3   tz           3440 non-null   object
 4   gr           2919 non-null   object
 5   g            3440 non-null   object
 6   h            3440 non-null   object
 7   l            3440 non-null   object
 8   al           3094 non-null   object
 9   hh           3440 non-null   object
 10  r            3440 non-null   object
 11  u            3440 non-null   object
 12  t            3440 non-null   float64
 13  hc           3440 non-null   float64
 14  cy           2919 non-null   object
 15  ll           2919 non-null   object
 16  _heartbeat_  120 non-null    float64
 17  kw           93 non-null     object
dtypes: float64(4), object(14)
memory usage: 500.8+ KB
0     America/New_York
1       America/Denver
2     America/New_York
3    America/Sao_Paulo
4     America/New_York
Name: tz, dtype: object
```

Hình 1. 8 Những thông tin cơ bản của tập Dataframe

Phương thức .value_counts() của Series pandas được áp dụng trực tiếp lên cột múi giờ (frame["tz"]). Phương thức này là công cụ tối ưu trong pandas để tính toán tần suất xuất hiện của các giá trị duy nhất (unique values), tự động sắp xếp kết quả theo thứ tự giảm dần của tần suất.

```
tz_counts = frame["tz"].value_counts()
tz_counts.head()
```

```
tz
America/New_York       1251
                        521
America/Chicago         400
America/Los_Angeles     382
America/Denver          191
Name: count, dtype: int64
```

Hình 1. 9 Tần suất của TimeZones dựa trên pandas

Kết quả đếm bằng pandas hoàn toàn khớp với kết quả từ Counter của Python, xác nhận America/New_York (1251) và chuỗi trống (521) là các giá trị đứng đầu. Sự nhất quán này củng cố tính chính xác của dữ

liệu và thiết lập cột tz làm cơ sở cho các phân tích dữ liệu đa chiều tiếp theo.

Để đảm bảo tính chính xác và thẩm mỹ của biểu đồ, dữ liệu múi giờ cần phải trải qua quá trình làm sạch để xử lý các giá trị bị thiếu và không xác định.

Trong DataFrame, những bản ghi ban đầu không có trường tz được pandas gán giá trị NaN. Các giá trị này được thay thế bằng chuỗi "Missing" bằng cách sử dụng phương thức fillna trên cột múi giờ.

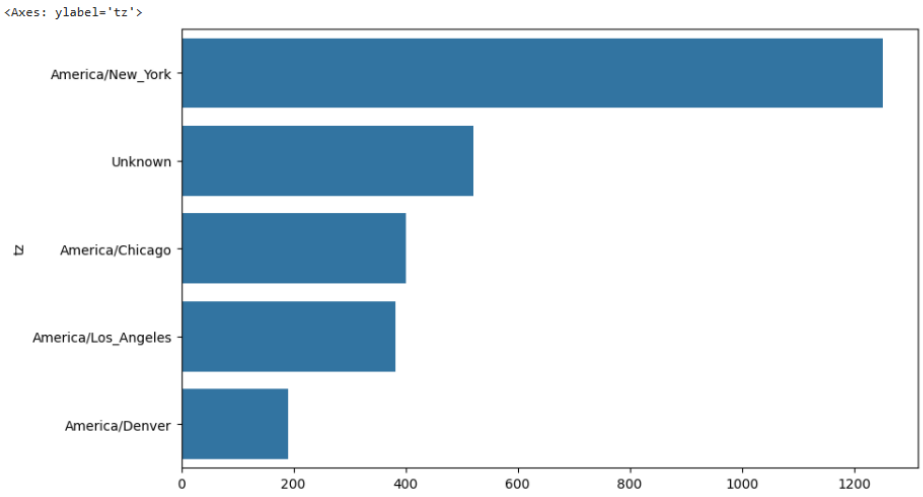
Các chuỗi trống " đại diện cho những trường hợp mà trường tz tồn tại nhưng không thể xác định được múi giờ. Để phân biệt chúng với các giá trị NaN đã được điền, các chuỗi trống này được thay thế bằng chuỗi "Unknown" bằng cách sử dụng chỉ mục mảng Boolean (Boolean array indexing). Sau khi hoàn thành, cột múi giờ đã được làm sạch (clean_tz) được sử dụng để tính toán lại tần suất đếm cuối cùng (tz_counts).

```
clean_tz = frame["tz"].fillna("Missing")
clean_tz[clean_tz == ""] = "Unknown"
tz_counts = clean_tz.value_counts()
tz_counts.head()
```

```
tz
America/New_York       1251
Unknown                 521
America/Chicago         400
America/Los_Angeles     382
America/Denver          191
Name: count, dtype: int64
```

Hình 1. 10 Dữ liệu sau khi được làm sạch

Cuối cùng, dữ liệu được trực quan hóa. Việc này thường được thực hiện bằng cách lấy một tập hợp con, chẳng hạn như 5 hoặc 10 múi giờ hàng đầu, và sử dụng gói seaborn để tạo biểu đồ thanh ngang (barplot). Biểu đồ này cung cấp một cái nhìn rõ ràng và trực quan về sự phân bố địa lý của người dùng, làm nổi bật các khu vực truy cập chính và tỷ lệ các bản ghi không thể xác định được múi giờ.



Hình 1. 11 Biểu đồ top 10 tần suất TimeZones hàng đầu của dữ liệu

3. Phân tích hành vi của người dùng theo hệ điều hành

Trường a (chuỗi tác nhân) là một chuỗi văn bản chứa thông tin chi tiết về trình duyệt, thiết bị hoặc ứng dụng mà người dùng đã sử dụng để truy cập và rút gọn URL. Phân tích trường này cho phép khám phá hành vi và đặc điểm công nghệ của người dùng.

Việc phân tích (parsing) toàn bộ thông tin thú vị trong các chuỗi "agent" (tức là User-Agent strings) có thể trông khá phức tạp — vì mỗi chuỗi chứa nhiều thông tin chi tiết về trình duyệt, hệ điều hành, thiết bị, phiên bản, v.v.

Một chiến lược khả thi để đơn giản hóa là tách (split) token đầu tiên trong chuỗi. Phần này thường biểu thị loại trình duyệt hoặc khả năng của trình duyệt (browser capability) một cách sơ bộ. Sau đó, ta có thể tạo một bảng tóm tắt (summary) để phân tích hành vi người dùng dựa trên loại trình duyệt/nền tảng này.

Quá trình này bao gồm ba bước chính:

Làm sạch: Loại bỏ các giá trị thiếu (NaN) khỏi cột a bằng phương thức dropna() để đảm bảo chỉ xử lý các chuỗi tác nhân hợp lệ.

Tách chuỗi: Sử dụng list comprehension kết hợp với phương thức split() để lấy token đầu tiên của mỗi chuỗi tác nhân.

Đếm: Kết quả trích xuất là một Series mới (results), được sử dụng để đếm tần suất xuất hiện của các token này.

```
Mozilla/5.0                2594
Mozilla/4.0                601
GoogleMaps/RochesterNY     121
Opera/9.80                 34
TEST_INTERNET_AGENT        24
GoogleProducer             21
Mozilla/6.0                5
BlackBerry8520/5.0.0.681   4
Name: count, dtype: int64
```

Hình 1. 12 Một số token phổ biến trong dữ liệu

Phân tích cho thấy các token phổ biến nhất là Mozilla/5.0 (2594 lượt), thường đại diện cho các trình duyệt hiện đại, và Mozilla/4.0 (601 lượt), thường liên quan đến các phiên bản trình duyệt cũ hơn hoặc môi trường không phải trình duyệt truyền thống.

Để phân tích sâu hơn về sự khác biệt hành vi giữa các nền tảng, ta tiến hành phân tách 10 múi giờ hàng đầu thành nhóm người dùng Windows và nhóm Non-Windows.

Bước đầu tiên là tạo một DataFrame mới, gọi là cframe, chỉ chứa các bản ghi không bị thiếu giá trị trong cả trường tác nhân (a) và trường múi giờ (tz), đảm bảo tính toàn vẹn của dữ liệu phân tích.

Một cột mới tên là "os" được thêm vào DataFrame. Logic phân loại dựa trên sự xuất hiện của chuỗi "Windows" trong chuỗi tác nhân (cframe["a"]). Nếu chuỗi "Windows" xuất hiện, người dùng được gán nhãn "Windows". Ngược lại, họ được gán nhãn "Not Windows". Việc này được thực hiện hiệu quả bằng hàm numpy.where để gán giá trị có điều kiện.

```
0        Windows
1    Not Windows
2        Windows
3    Not Windows
4        Windows
Name: os, dtype: object
```

Hình 1. 13 Phân loại hệ điều hành trong bộ dữ liệu

Dữ liệu sau đó được nhóm theo hai tiêu chí: múi giờ (tz) và hệ điều hành (os). Tiếp theo, phương thức size() được sử dụng để tính kích thước (số lần đếm) của mỗi nhóm tổ hợp múi giờ/hệ điều hành.

Phương thức unstack() được áp dụng để chuyển dữ liệu đếm từ dạng cột (Series) thành dạng bảng tổng hợp (pivot table), được đặt tên là agg_counts. Bảng này có các múi giờ là chỉ mục (index) và hai loại OS ("Windows" và "Not Windows") là các cột. Các giá trị thiếu do tổ hợp múi giờ/OS không tồn tại được điền bằng 0 (fillna(0)).

os	Not Windows	Windows
tz		
	245.0	276.0
Africa/Cairo	0.0	3.0
Africa/Casablanca	0.0	1.0
Africa/Ceuta	0.0	2.0
Africa/Johannesburg	0.0	1.0

Hình 1. 14 Dữ liệu sau khi được nhóm theo tz và os

Để tập trung phân tích vào các múi giờ hoạt động nhất, ta cần xác định và trực quan hóa 10 múi giờ có tổng lượt truy cập cao nhất.

Việc này được thực hiện bằng cách tạo một mảng chỉ số gián tiếp (indirect index array) dựa trên tổng số bản ghi của từng múi giờ. Đầu tiên, tính tổng số bản ghi của từng múi giờ bằng cách gọi agg_counts.sum(1) (hoặc agg_counts.sum("columns")).

```
indexer = agg_counts.sum("columns").argsort()
indexer.values[:10]
```

```
array([ 2,  7,  5,  4, 14, 11,  9,  8, 24, 26])
```

Hình 1. 15 Tổng số bản ghi của từng TimeZones

Tiếp theo, ta có thể sử dụng phương thức argsort() để lấy ra một mảng chỉ số sắp xếp các giá trị theo thứ tự tăng dần.

os	Not Windows	Windows
tz		
America/Sao_Paulo	13.0	20.0
Europe/Madrid	16.0	19.0
Pacific/Honolulu	0.0	36.0
Asia/Tokyo	2.0	35.0
Europe/London	43.0	31.0
America/Denver	132.0	59.0
America/Los_Angeles	130.0	252.0
America/Chicago	115.0	285.0
	245.0	276.0
America/New_York	339.0	912.0

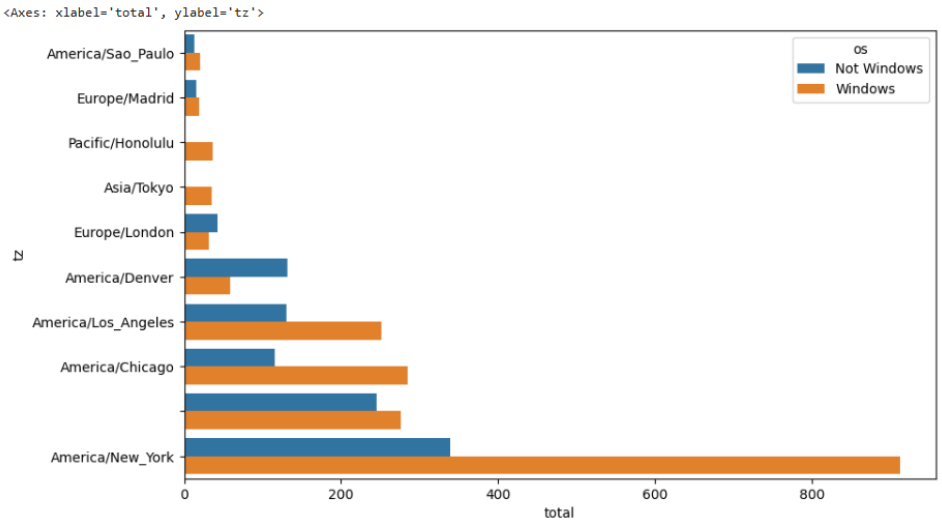
Hình 1. 16 Bảng dữ liệu sau khi cắt bỏ 10 chỉ số cuối cùng (múi giờ lớn nhất)

Sau đó, 10 múi giờ lớn nhất được chọn bằng cách lấy 10 chỉ số cuối cùng (hoặc sử dụng phương thức nlargest(10) của pandas một cách tiện lợi hơn).

```
tz
America/New_York       1251.0
                        521.0
America/Chicago         400.0
America/Los_Angeles     382.0
America/Denver          191.0
Europe/London            74.0
Asia/Tokyo               37.0
Pacific/Honolulu         36.0
Europe/Madrid            35.0
America/Sao_Paulo        33.0
dtype: float64
```

Hình 1. 17 Sử dụng take để chọn các hàng theo thứ tự (múi giờ lớn nhất)

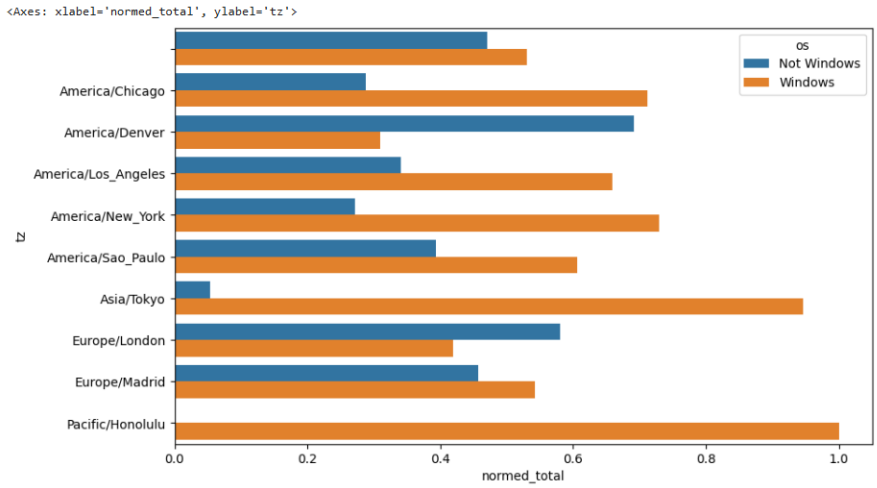
Tập hợp con được chọn này (count_subset) được định hình lại. Dữ liệu được chuyển từ dạng cột kép (unstack) về dạng hàng đơn (stack()). Sử dụng reset_index() để chuyển các chỉ mục thành các cột dữ liệu thông thường. Cuối cùng, biểu đồ thanh nhóm được tạo bằng gói seaborn với tham số hue="os", trực quan hóa số lượng tuyệt đối của người dùng Windows so với Non-Windows trong 10 múi giờ hàng đầu.



Hình 1. 18 Biểu đồ số lượng người dùng Windows và Non-Windows trong từng múi giờ.

Để thấy rõ tỷ lệ phần trăm người dùng Windows so với Non-Windows trong từng múi giờ, dữ liệu cần được chuẩn hóa (normalized). Việc này đặc biệt hữu ích khi so sánh các nhóm múi giờ có tổng số lượt truy cập khác nhau, cho phép đánh giá mức độ phổ biến tương đối của mỗi hệ điều hành.

Quá trình chuẩn hóa được thực hiện bằng cách định nghĩa một hàm tính tỷ lệ phần trăm của mỗi loại OS so với tổng số lần đếm trong nhóm múi giờ đó. Hàm chuẩn hóa này sau đó được áp dụng lên dữ liệu đã nhóm theo múi giờ thông qua phương thức groupby("tz").apply(norm_total), tạo ra cột normed_total chứa tỷ lệ phần trăm đã được chuẩn hóa. Cuối cùng, biểu đồ thanh được sử dụng với cột normed_total này để trực quan hóa tỷ lệ phần trăm người dùng Windows và Non-Windows. Phân tích này cho phép so sánh tỷ lệ thống trị tương đối của hệ điều hành giữa các khu vực địa lý khác nhau một cách khách quan.

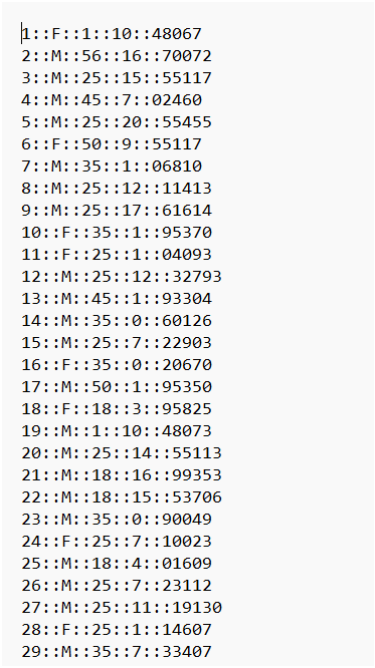


Hình 1. 19 Biểu đồ trực quan hóa tỷ lệ phần trăm người dùng Windows và Non-Windows

Chương 3: Phân tích Bộ dữ liệu MovieLens 1M

1. Tổng quan về bộ dữ liệu

Bộ dữ liệu MovieLens 1M được cung cấp bởi GroupLens Research, là một tập hợp các xếp hạng phim được thu thập từ những người dùng tích cực vào cuối thập niên 1990 và đầu thập niên 2000. Đây là một bộ dữ liệu quan trọng trong nghiên cứu hệ thống khuyến nghị, chứa một triệu xếp hạng (1 Million ratings) được thu thập từ khoảng 6.000 người dùng trên 4.000 phim.



```
1::F::1::10::48067
2::M::56::16::70072
3::M::25::15::55117
4::M::45::7::02460
5::M::25::20::55455
6::F::50::9::55117
7::M::35::1::06810
8::M::25::12::11413
9::M::25::17::61614
10::F::35::1::95370
11::F::25::1::04093
12::M::25::12::32793
13::M::45::1::93304
14::M::35::0::60126
15::M::25::7::22903
16::F::35::0::20670
17::M::50::1::95350
18::F::18::3::95825
19::M::1::10::48073
20::M::25::14::55113
21::M::18::16::99353
22::M::18::15::53706
23::M::35::0::90049
24::F::25::7::10023
25::M::18::4::01609
26::M::25::7::23112
27::M::25::11::19130
28::F::25::1::14607
29::M::35::7::33407
```

Hình 2. 1 Bộ dữ liệu ban đầu của users.dat

```
1::Toy Story (1995)::Animation|Children's|Comedy
2::Jumanji (1995)::Adventure|Children's|Fantasy
3::Grumpier Old Men (1995)::Comedy|Romance
4::Waiting to Exhale (1995)::Comedy|Drama
5::Father of the Bride Part II (1995)::Comedy
6::Heat (1995)::Action|Crime|Thriller
7::Sabrina (1995)::Comedy|Romance
8::Tom and Huck (1995)::Adventure|Children's
9::Sudden Death (1995)::Action
10::GoldenEye (1995)::Action|Adventure|Thriller
11::American President, The (1995)::Comedy|Drama|Romance
12::Dracula: Dead and Loving It (1995)::Comedy|Horror
13::Balto (1995)::Animation|Children's
14::Nixon (1995)::Drama
15::Cutthroat Island (1995)::Action|Adventure|Romance
16::Casino (1995)::Drama|Thriller
17::Sense and Sensibility (1995)::Drama|Romance
18::Four Rooms (1995)::Thriller
19::Ace Ventura: When Nature Calls (1995)::Comedy
20::Money Train (1995)::Action
21::Get Shorty (1995)::Action|Comedy|Drama
22::Copycat (1995)::Crime|Drama|Thriller
23::Assassins (1995)::Thriller
24::Powder (1995)::Drama|Sci-Fi
25::Leaving Las Vegas (1995)::Drama|Romance
26::Othello (1995)::Drama
27::Now and Then (1995)::Drama
28::Persuasion (1995)::Romance
29::City of Lost Children, The (1995)::Adventure|Sci-Fi
```

Hình 2. 2 Bộ dữ liệu ban đầu của movies.dat

```
1::1193::5::978300760
1::661::3::978302109
1::914::3::978301968
1::3408::4::978300275
1::2355::5::978824291
1::1197::3::978302268
1::1287::5::978302039
1::2804::5::978300719
1::594::4::978302268
1::919::4::978301368
1::595::5::978824268
1::938::4::978301752
1::2398::4::978302281
1::2918::4::978302124
1::1035::5::978301753
1::2791::4::978302188
1::2687::3::978824268
1::2018::4::978301777
1::3105::5::978301713
1::2797::4::978302039
1::2321::3::978302205
1::720::3::978300760
1::1270::5::978300055
1::527::5::978824195
1::2340::3::978300103
1::48::5::978824351
1::1097::4::978301953
1::1721::4::978300055
1::1545::4::978824139
```

Hình 2. 3 Bộ dữ liệu ban đầu của ratings.dat

Dữ liệu được phân tán trên ba bảng riêng biệt, mỗi bảng được tải vào một đối tượng DataFrame của thư viện pandas để dễ dàng thao tác và phân tích:

Thông tin Người dùng (users): Bảng này chứa các đặc điểm cá nhân của người dùng, bao gồm ID người dùng (user_id), giới tính (gender), độ tuổi (age), nghề nghiệp (occupation), và mã zip (zip code).

Cần lưu ý rằng các trường độ tuổi và nghề nghiệp được mã hóa dưới dạng số nguyên, biểu thị các nhóm được định nghĩa trong tài liệu README đi kèm bộ dữ liệu.

Xếp hạng (ratings): Bảng này chứa các tương tác xếp hạng của người dùng đối với các bộ phim, bao gồm ID người dùng, ID phim (movie_id), xếp hạng (rating) (từ 1 đến 5 sao), và dấu thời gian (timestamp) của xếp hạng.

Thông tin Phim (movies): Bảng này chứa các thông tin mô tả về các bộ phim, bao gồm ID phim, tiêu đề (title), và thể loại (genres) của bộ phim.

Các bảng này được đọc vào pandas bằng hàm pandas.read_table, sử dụng ký tự :: làm dấu phân cách và thiết lập tham số header=None do các tệp dữ liệu không chứa hàng tiêu đề.

	user_id	gender	age	occupation	zip
0	1	F	1	10	48067
1	2	M	56	16	70072
2	3	M	25	15	55117
3	4	M	45	7	02460
4	5	M	25	20	55455

Hình 2. 4 Bảng chứa dữ liệu users(5 dòng đầu)

	user_id	movie_id	rating	timestamp
0	1	1193	5	978300760
1	1	661	3	978302109
2	1	914	3	978301968
3	1	3408	4	978300275
4	1	2355	5	978824291

Hình 2. 5 Bảng chứa dữ liệu ratings (5 dòng đầu)

	movie_id	title	genres
0	1	Toy Story (1995)	Animation|Children's|Comedy
1	2	Jumanji (1995)	Adventure|Children's|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama
4	5	Father of the Bride Part II (1995)	Comedy

Hình 2. 6 Bảng chứa dữ liệu movies (5 dòng đầu)

2. Hợp nhất dữ liệu (Merging Data)

Việc phân tích dữ liệu trải rộng trên ba bảng riêng biệt là một nhiệm vụ phức tạp, đòi hỏi phải tích hợp thông tin từ các nguồn khác nhau. Ví dụ: để tính toán xếp hạng trung bình của một bộ phim theo giới tính và độ tuổi, cần phải kết hợp thông tin từ bảng Xếp hạng, Người dùng, và Phim.

Để đơn giản hóa quá trình phân tích đa chiều này, tất cả dữ liệu được hợp nhất thành một bảng duy nhất (DataFrame) bằng cách sử dụng hàm pandas.merge.

Đây là nội dung cho mục Hợp nhất Dữ liệu (Merging Data), được trình bày dưới dạng báo cáo:

Quá trình hợp nhất được thực hiện theo hai bước tuần tự. Thứ nhất, h nhất bảng ratings với bảng users dựa trên khóa chung là user_id. Sau đó, hợp nhất kết quả từ tạo với bảng movies dựa trên khóa chung là movie_id. Trong quá trình này, pandas tự động suy luận các cột chung (user_id và movie_id) để sử dụng làm khóa hợp nhất (join keys). Quá trình này tạo ra một DataFrame lớn, chứa tổng cộng 1.000.209 hàng và 10 cột, bao gồm tất cả các thông tin cần thiết để tiến hành phân tích sâu rộng.

	user_id	movie_id	rating	timestamp	gender	age	occupation	zip	title	genres
0	1	1193	5	978300760	F	1	10	48067	One Flew Over the Cuckoo's Nest (1975)	Drama
1	1	661	3	978302109	F	1	10	48067	James and the Giant Peach (1996)	Animation|Children's|Musical
2	1	914	3	978301968	F	1	10	48067	My Fair Lady (1964)	Musical|Romance
3	1	3408	4	978300275	F	1	10	48067	Erin Brockovich (2000)	Drama
4	1	2355	5	978824291	F	1	10	48067	Bug's Life, A (1998)	Animation|Children's|Comedy
...	...	...	...	...	...	...	...	...	...	...
1000204	6040	1091	1	956716541	M	25	6	11106	Weekend at Bernie's (1989)	Comedy
1000205	6040	1094	5	956704887	M	25	6	11106	Crying Game, The (1992)	Drama|Romance|War
1000206	6040	562	5	956704746	M	25	6	11106	Welcome to the Dollhouse (1995)	Comedy|Drama
1000207	6040	1096	4	956715648	M	25	6	11106	Sophie's Choice (1982)	Drama
1000208	6040	1097	4	956715569	M	25	6	11106	E.T. the Extra-Terrestrial (1982)	Children's|Drama|Fantasy|Sci-Fi

1000209 rows × 10 columns

Hình 2. 7 Bảng dữ liệu sau khi đã gộp 3 bảng với nhau

3. Xếp hạng Trung bình theo Tiêu đề phim và Giới tính

Sau khi quá trình hợp nhất dữ liệu hoàn tất, ta có thể dễ dàng tính toán các thống kê tổng hợp (aggregations), chẳng hạn như xếp hạng trung bình, bằng cách nhóm dữ liệu theo các tiêu chí quan tâm.

Để lấy xếp hạng trung bình cho mỗi phim được nhóm theo giới tính, ta sử dụng phương thức pivot_table của pandas. Phương thức này cho phép tổ chức lại dữ liệu và áp dụng một hàm tổng hợp một cách linh hoạt: Cột rating (xếp hạng) là giá trị cần tính trung bình, Cột title (tiêu đề phim) được sử dụng làm nhãn hàng, Cột gender (giới tính) được sử dụng để tạo ra các cột riêng biệt (F - Nữ, M - Nam), Hàm mean được sử dụng để tính giá trị trung bình. Kết quả thu được là một DataFrame mới (mean_ratings) với tiêu đề phim làm nhãn hàng và giới tính làm nhãn cột, hiển thị xếp hạng trung bình của mỗi phim được phân tách theo giới tính.

gender	F	M
title		
$1,000,000 Duck (1971)	3.375000	2.761905
'Night Mother (1986)	3.388889	3.352941
'Til There Was You (1997)	2.675676	2.733333
'burbs, The (1989)	2.793478	2.962085
...And Justice for All (1979)	3.828571	3.689024

Hình 2. 8 Điểm đánh giá trung bình của mỗi bộ phim theo giới tính

Để tránh các kết quả bị sai lệch do phim có quá ít đánh giá, cần lọc ra các bộ phim nhận được ít nhất 250 lượt đánh giá (số này mang tính tùy ý). Để làm điều này, ta nhóm dữ liệu theo cột title và dùng phương thức size() để lấy một Series cho biết kích thước của mỗi nhóm (tức số lượt đánh giá của từng phim).

Nhóm dữ liệu (data) theo title và sử dụng size() để lấy Series chứa số lượng xếp hạng cho mỗi tiêu đề (ratings_by_title). Sử dụng chỉ mục boolean để chọn các tiêu đề có số lượng xếp hạng lớn hơn hoặc bằng 250, tạo ra danh sách các tiêu đề phim hoạt động (active_titles).

```
Index(['''burbs, The (1989)', '10 Things I Hate About You (1999)',
       '101 Dalmatians (1961)', '101 Dalmatians (1996)', '12 Angry Men (1957)',
       '13th Warrior, The (1999)', '2 Days in the Valley (1996)',
       '20,000 Leagues Under the Sea (1954)', '2001: A Space Odyssey (1968)',
       '2010 (1984)',
       ...
       'X-Men (2000)', 'Year of Living Dangerously (1982)',
       'Yellow Submarine (1968)', 'You've Got Mail (1998)',
       'Young Frankenstein (1974)', 'Young Guns (1988)',
       'Young Guns II (1990)', 'Young Sherlock Holmes (1985)',
       'Zero Effect (1998)', 'eXistenZ (1999)'],
      dtype='object', name='title', length=1216)
```

Hình 2. 9 Các bộ phim nhận được ít nhất 250 lượt đánh giá

Sử dụng phương thức .loc để chọn các hàng tương ứng từ DataFrame mean_ratings, giới hạn phân tích chỉ trong các phim hoạt động.

gender	F	M
title		
'burbs, The (1989)	2.793478	2.962085
10 Things I Hate About You (1999)	3.646552	3.311966
101 Dalmatians (1961)	3.791444	3.500000
101 Dalmatians (1996)	3.240000	2.911215
12 Angry Men (1957)	4.184397	4.328421
...	...	...
Young Guns (1988)	3.371795	3.425620
Young Guns II (1990)	2.934783	2.904025
Young Sherlock Holmes (1985)	3.514706	3.363344
Zero Effect (1998)	3.864407	3.723140
eXistenZ (1999)	3.098592	3.289086

1216 rows × 2 columns

Hình 2. 10 Lọc các hàng tương ứng từ DataFrame của các bộ phim

4. Đo lường Sự khác biệt về Xếp hạng

Phân tích không chỉ giới hạn ở việc tính toán xếp hạng trung bình mà còn mở rộng sang việc đo lường sự khác biệt về sở thích và mức độ bất đồng giữa những người xem.

Sau khi lọc và sắp xếp DataFrame mean_ratings (chứa xếp hạng trung bình của phim hoạt động), ta có thể xác định các bộ phim được ưa thích nhất theo giới tính.

gender	F	M
title		
24 7: Twenty Four Seven (1997)	5.0	3.750000
Gate of Heavenly Peace, The (1995)	5.0	5.000000
Big Combo, The (1955)	5.0	3.600000
Dancemaker (1998)	5.0	3.250000
Clean Slate (Coup de Torchon) (1981)	5.0	3.857143

Hình 2. 11 Các bộ phim được nữ khán giả đánh giá cao nhất

Sắp xếp DataFrame mean_ratings theo cột **F** (xếp hạng của nữ) theo thứ tự giảm dần (ascending=False). Ví dụ: các phim như *Close Shave, A (1995)*, *Wrong Trousers, The (1993)* và *Sunset Blvd. (a.k.a. Sunset Boulevard) (1950)* nằm trong số các phim có xếp hạng cao nhất từ khán giả nữ.

Để tìm phim có sự khác biệt lớn nhất giữa nam và nữ, một cột mới "diff" được tạo bằng cách lấy xếp hạng trung bình của nam trừ đi xếp hạng trung bình của nữ (mean_ratings["M"] - mean_ratings["F"]). Sắp xếp theo cột diff cho thấy các phim được Phụ nữ ưa thích hơn (giá trị khác biệt âm lớn nhất). Ví dụ: Dirty Dancing (1987) có mức khác biệt là -0.830.

gender	F	M	diff
title			
Dirty Dancing (1987)	3.790378	2.959596	-0.830782
Jumpin' Jack Flash (1986)	3.254717	2.578358	-0.676359
Grease (1978)	3.975265	3.367041	-0.608224
Little Women (1994)	3.870588	3.321739	-0.548849
Steel Magnolias (1989)	3.901734	3.365957	-0.535777

Hình 2. 12 Các phim được Phụ nữ ưa thích hơn xếp theo cột diff

Sắp xếp ngược lại (sorted_by_diff[::-1]) cho thấy các phim được Đàn ông ưa thích hơn (giá trị khác biệt dương lớn nhất). Ví dụ: Good, The Bad and The Ugly, The (1966) có mức khác biệt là 0.726.

gender	F	M	diff
title			
Good, The Bad and The Ugly, The (1966)	3.494949	4.221300	0.726351
Kentucky Fried Movie, The (1977)	2.878788	3.555147	0.676359
Dumb & Dumber (1994)	2.697987	3.336595	0.638608
Longest Day, The (1962)	3.411765	4.031447	0.619682
Cable Guy, The (1996)	2.250000	2.863787	0.613787

Hình 2. 13 Các phim được Đàn ông ưa thích xếp theo cột diff

Để tìm ra những bộ phim gây tranh cãi nhất (controversial), độc lập với giới tính, ta có thể đo lường mức độ bất đồng bằng cách tính toán độ lệch chuẩn (standard deviation - std) của các xếp hạng. Độ lệch chuẩn phản ánh sự phân tán (variance) của xếp hạng xung quanh giá trị trung bình.

Trước tiên ta cần phải nhóm dữ liệu xếp hạng theo cột title và tính độ lệch chuẩn của cột rating thông qua phương thức std(). Sau đó lọc chỉ mục active_titles (phim có ít nhất 250 xếp hạng) được áp dụng để đảm bảo độ lệch chuẩn chỉ được tính cho các bộ phim hoạt động.

```
title
'burbs, The (1989)                  1.107760
10 Things I Hate About You (1999)   0.989815
101 Dalmatians (1961)               0.982103
101 Dalmatians (1996)               1.098717
12 Angry Men (1957)                 0.812731
Name: rating, dtype: float64
```

Hình 2. 14 Tính độ lệch chuẩn sự phân tán xung quanh

Kết quả được sắp xếp theo thứ tự giảm dần (ascending=False).

```
title
Dumb & Dumber (1994)                     1.321333
Blair Witch Project, The (1999)          1.316368
Natural Born Killers (1994)              1.307198
Tank Girl (1995)                         1.277695
Rocky Horror Picture Show, The (1975)    1.260177
Eyes Wide Shut (1999)                    1.259624
Evita (1996)                             1.253631
Billy Madison (1995)                     1.249970
Fear and Loathing in Las Vegas (1998)    1.246408
Bicentennial Man (1999)                  1.245533
Name: rating, dtype: float64
```

Hình 2. 15 Phim có sự khác biệt về đánh giá lớn

Các phim có độ lệch chuẩn cao nhất (ví dụ: *Dumb & Dumber (1994)*, *Blair Witch Project, The (1999)*) được coi là những bộ phim gây bất đồng sâu sắc nhất giữa những người xem, vì có sự phân tán lớn giữa các xếp hạng cao và thấp.

5. Phân tích Xếp hạng theo Thể loại (Genre) và Độ tuổi

Một thách thức trong việc phân tích sở thích theo thể loại là một bộ phim có thể thuộc về nhiều thể loại khác nhau, được lưu trữ dưới dạng chuỗi với các thể loại phân cách bằng dấu gạch đứng (|). Để phân tích từng thể loại một cách độc lập, cần phải xử lý cột genre để tạo ra các bản ghi riêng lẻ cho mỗi thể loại.

```
0     Animation|Children's|Comedy
1    Adventure|Children's|Fantasy
2                  Comedy|Romance
3                    Comedy|Drama
4                          Comedy
Name: genres, dtype: object
```

Hình 2. 16 Bộ dữ liệu cột genre ban đầu

Quá trình này bao gồm hai bước chính: Sử dụng phương thức str.split("|") trên cột genres của DataFrame movies để chuyển các chuỗi thể loại thành một danh sách (list) các thể loại.

	movie_id	title	genre
0	1	Toy Story (1995)	[Animation, Children's, Comedy]
1	2	Jumanji (1995)	[Adventure, Children's, Fantasy]
2	3	Grumpier Old Men (1995)	[Comedy, Romance]
3	4	Waiting to Exhale (1995)	[Comedy, Drama]
4	5	Father of the Bride Part II (1995)	[Comedy]

Hình 2. 17 Tách dữ liệu của cột genres sử dụng split()

Sau khi chuyển đổi, phương thức movies.explode("genres") được gọi. Thao tác này tạo ra một DataFrame mới (movies_exploded) trong đó mỗi hàng chỉ chứa một thể loại duy nhất cho mỗi bộ phim. Ví dụ, nếu một phim được gán ba thể loại, nó sẽ được sao chép và xuất hiện ba lần trong DataFrame mới, mỗi lần với một thể loại khác nhau.

	movie_id	title	genre
0	1	Toy Story (1995)	Animation
0	1	Toy Story (1995)	Children's
0	1	Toy Story (1995)	Comedy
1	2	Jumanji (1995)	Adventure
1	2	Jumanji (1995)	Children's
1	2	Jumanji (1995)	Fantasy
2	3	Grumpier Old Men (1995)	Comedy
2	3	Grumpier Old Men (1995)	Romance
3	4	Waiting to Exhale (1995)	Comedy
3	4	Waiting to Exhale (1995)	Drama

Hình 2. 18 Bảng dữ liệu cho từng thể loại từng bộ phim

Sau khi dữ liệu thể loại đã được mở rộng, ta có thể tính toán xếp hạng trung bình theo thể loại và nhóm tuổi. Hợp nhất DataFrame movies_exploded với bảng xếp hạng (ratings) và thông tin người dùng (users) để tạo ra một DataFrame cuối cùng (ratings_with_genre). DataFrame này chứa xếp hạng và thông tin người dùng cho mỗi thể loại riêng lẻ, tạo cơ sở cho việc phân tích đa chiều.

```
movie_id                    1
title         Toy Story (1995)
genre                Animation
user_id                      1
rating                       5
timestamp            978824268
gender                       F
age                          1
occupation                  10
zip                      48067
Name: 0, dtype: object
```

Hình 2. 19 Dữ liệu sau khi đã hợp nhất

Sử dụng phương thức groupby để tính xếp hạng trung bình (mean()) theo hai tiêu chí: genres và age. Sau đó, sử dụng phương thức unstack("age") để chuyển các nhóm tuổi (1, 18, 25, 35, 45, 50, 56) thành các cột. Kết quả là một bảng tổng hợp trực quan, giúp dễ dàng so sánh xếp hạng trung bình của từng thể loại phim giữa các nhóm tuổi khác nhau. Ví dụ, thể loại Film-Noir có xếp hạng trung bình cao nhất (4.145) trong nhóm tuổi 1 (dưới 18 tuổi).

age	1	18	25	35	45	50	56
genre							
Action	3.506385	3.447097	3.453358	3.538107	3.528543	3.611333	3.610709
Adventure	3.449975	3.408525	3.443163	3.515291	3.528963	3.628163	3.649064
Animation	3.476113	3.624014	3.701228	3.740545	3.734856	3.780020	3.756233
Children's	3.241642	3.294257	3.426873	3.518423	3.527593	3.556555	3.621822
Comedy	3.497491	3.460417	3.490385	3.561984	3.591789	3.646868	3.650949
Crime	3.710170	3.668054	3.680321	3.733736	3.750661	3.810688	3.832549
Documentary	3.730769	3.865865	3.946690	3.953747	3.966521	3.908108	3.961538
Drama	3.794735	3.721930	3.726428	3.782512	3.784356	3.878415	3.933465
Fantasy	3.317647	3.353778	3.452484	3.482301	3.532468	3.581570	3.532700
Film-Noir	4.145455	3.997368	4.058725	4.064910	4.105376	4.175401	4.125932

Hình 2. 20 Xếp hạng trung bình của từng thể loại phim giữa các nhóm tuổi khác nhau

Chương 4: Xu hướng Tên Trẻ Sơ Sinh ở Mỹ 1880-2010

1. Tổng quan và chuẩn bị dữ liệu

Dữ liệu về tần suất tên trẻ sơ sinh được cung cấp bởi Cục An sinh Xã hội Hoa Kỳ (SSA), là một nguồn dữ liệu quan trọng và được sử dụng rộng rãi, chẳng hạn bởi Hadley Wickham (tác giả của một số gói R phổ biến), để minh họa việc thao tác dữ liệu. Các tệp dữ liệu gốc chứa tổng số ca sinh cho từng sự kết hợp giữa giới tính và tên.

Dữ liệu được thu thập từ Social Security Administration (SSA) của Hoa Kỳ. Các tệp thô có định dạng yobYYYY.txt (với YYYY là năm) và được lắp ráp từ năm 1880 đến 2010 thành một DataFrame duy nhất (names).

```
Mary,F,7065
Anna,F,2604
Emma,F,2003
Elizabeth,F,1939
Minnie,F,1746
Margaret,F,1578
Ida,F,1472
Alice,F,1414
Bertha,F,1320
Sarah,F,1288
Annie,F,1258
Clara,F,1226
Ella,F,1156
Florence,F,1063
Cora,F,1045
Martha,F,1040
Laura,F,1012
Nellie,F,995
Grace,F,982
Carrie,F,949
Maude,F,858
Mabel,F,808
Bessie,F,794
Jennie,F,793
Gertrude,F,787
Julia,F,783
Hattie,F,769
Edith,F,768
Mattie,F,704
Rose,F,700
Catherine,F,688
```

Hình 3. 1 Bộ dữ liệu ban đầu của Yob1880.txt

Dữ liệu chỉ bao gồm các tên xuất hiện ít nhất 5 lần (Count >= 5) trong một năm. Giới hạn này được đặt ra nhằm bảo vệ quyền riêng tư cá nhân; các tên xuất hiện ít hơn 5 lần sẽ không được ghi nhận.

	name	sex	births
0	Mary	F	7065
1	Anna	F	2604
2	Emma	F	2003
3	Elizabeth	F	1939
4	Minnie	F	1746
...	...	...	...
1995	Woodie	M	5
1996	Worthy	M	5
1997	Wright	M	5
1998	York	M	5
1999	Zachariah	M	5

2000 rows × 3 columns

Hình 3. 2 Dữ liệu sau khi được đọc và thêm vào bảng

DataFrame này chứa các cột: tên (name), giới tính (sex), số lượng tên sinh (births), và năm (year).

```
names1880.groupby("sex")["births"].sum()
```

```
sex
F     90993
M    110493
Name: births, dtype: int64
```

Hình 3. 3 Tổng số ca sinh theo giới tính và năm

Tiến hành gộp các file YOBYYYY.txt và thêm cột năm để có thể thực hiện xử lý một cách dễ dàng hơn.

	name	sex	births	year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
3	Elizabeth	F	1939	1880
4	Minnie	F	1746	1880
...	...	...	...	...
1690779	Zymaire	M	5	2010
1690780	Zyonne	M	5	2010
1690781	Zyquarius	M	5	2010
1690782	Zyran	M	5	2010
1690783	Zzyzx	M	5	2010

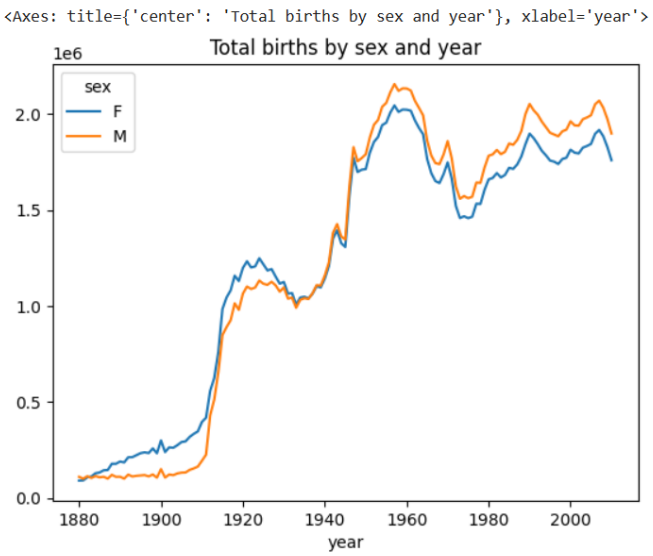
1690784 rows × 4 columns

Hình 3. 4 Dữ liệu sau khi gộp và thêm cột year

Sau khi tập hợp, dữ liệu được chuẩn hóa và tổng hợp: Tổng số ca sinh theo giới tính và năm được tính bằng cách sử dụng phương thức pivot_table.

sex	F	M
year		
2006	1896468	2050234
2007	1916888	2069242
2008	1883645	2032310
2009	1827643	1973359
2010	1759010	1898382

Hình 3. 5 Tổng hợp dữ liệu ở cấp độ year và sex



Hình 3. 6 Biểu đồ tổng số ca sinh theo giới tính và năm

Một cột mới, prop (tỷ lệ), đã được thêm vào. Cột này thể hiện phần trăm trẻ sơ sinh được đặt một cái tên cụ thể so với tổng số ca sinh của giới tính đó trong năm tương ứng. Giá trị này là rất quan trọng để phân tích sự phổ biến tương đối của một tên theo thời gian.

			name	sex	births	year	prop
year	**sex**						
1880	**F**	**0**	Mary	F	7065	1880	0.077643
		1	Anna	F	2604	1880	0.028618
		2	Emma	F	2003	1880	0.022013
		3	Elizabeth	F	1939	1880	0.021309
		4	Minnie	F	1746	1880	0.019188
...	...	...	...	...	...	...	...
2010	**M**	**1690779**	Zymaire	M	5	2010	0.000003
		1690780	Zyonne	M	5	2010	0.000003
		1690781	Zyquarius	M	5	2010	0.000003
		1690782	Zyran	M	5	2010	0.000003
		1690783	Zzyzx	M	5	2010	0.000003

1690784 rows × 5 columns

Hình 3. 7 Bảng dữ liệu phần trăm trẻ sơ sinh được đặt tên cụ thể

Để có thể nhìn nhận rõ hơn về dữ liệu cột prop, chúng ta cần xác minh rằng nó có thể cộng lại bằng 1 trong tất cả các nhóm.

```
year  sex
1880  F      1.0
      M      1.0
1881  F      1.0
      M      1.0
1882  F      1.0
            ...
2008  M      1.0
2009  F      1.0
      M      1.0
2010  F      1.0
      M      1.0
Name: prop, Length: 262, dtype: float64
```

Hình 3. 8 Xác minh dữ liệu đúng đắn

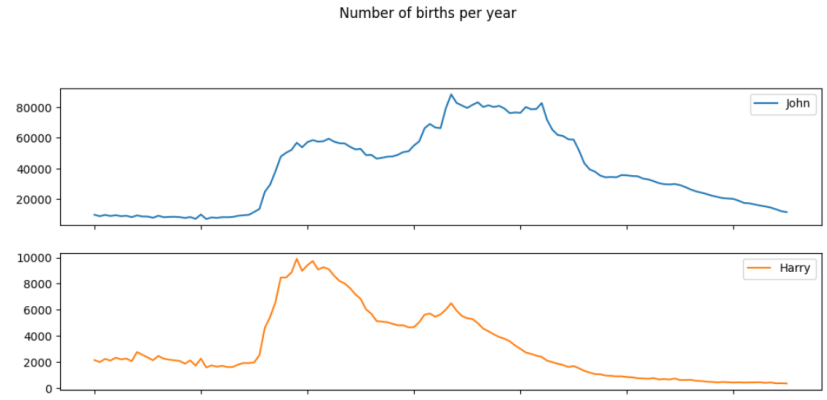
Để tạo điều kiện thuận lợi cho các phân tích tiếp theo, một tập hợp con (top1000) gồm 1.000 tên phổ biến nhất cho mỗi kết hợp giới tính/năm đã được trích xuất. Việc này giúp giảm kích thước dữ liệu mà vẫn giữ lại phần lớn thông tin về xu hướng.

			name	sex	births	year	prop
year	**sex**						
1880	**F**	**0**	Mary	F	7065	1880	0.077643
		1	Anna	F	2604	1880	0.028618
		2	Emma	F	2003	1880	0.022013
		3	Elizabeth	F	1939	1880	0.021309
		4	Minnie	F	1746	1880	0.019188

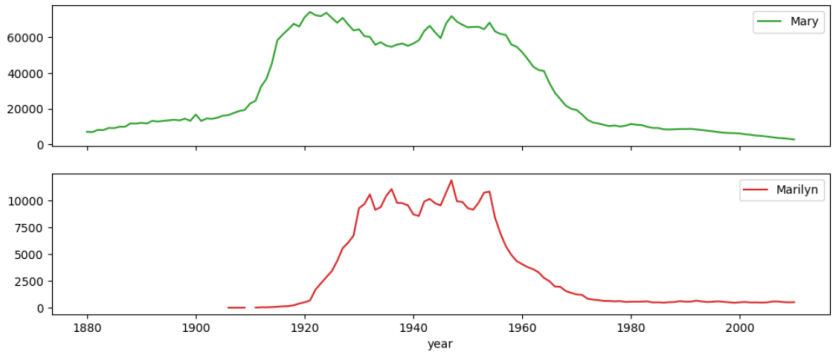
Hình 3. 9 Bảng dữ liệu sau khi lọc ra 1000 tên đặt nhiều nhất

2. Phân tích Xu hướng Đặt tên cụ thể

Việc phân tích xu hướng đặt tên bắt đầu bằng việc xem xét sự phổ biến của các tên riêng lẻ theo thời gian, sau đó mở rộng để đo lường tính đa dạng tổng thể.



Hình 3. 10 Biểu đồ của các tên truyền thống theo từng năm



Hình 3. 11 Biểu đồ của các tên truyền thống theo từng năm(2)

Việc vẽ biểu đồ chuỗi thời gian cho tỷ lệ xuất hiện (prop) của các tên cá nhân nổi bật như John, Harry, Mary, và Marilyn cho thấy rõ ràng các tên này đã giảm sự ưu ái trong dân số Mỹ qua nhiều thập kỷ. Để trực quan hóa điều này, một tập hợp con từ DataFrame top1000 được tạo ra, sau đó được định hình lại bằng pivot_table để có thể vẽ biểu đồ chuỗi thời gian.

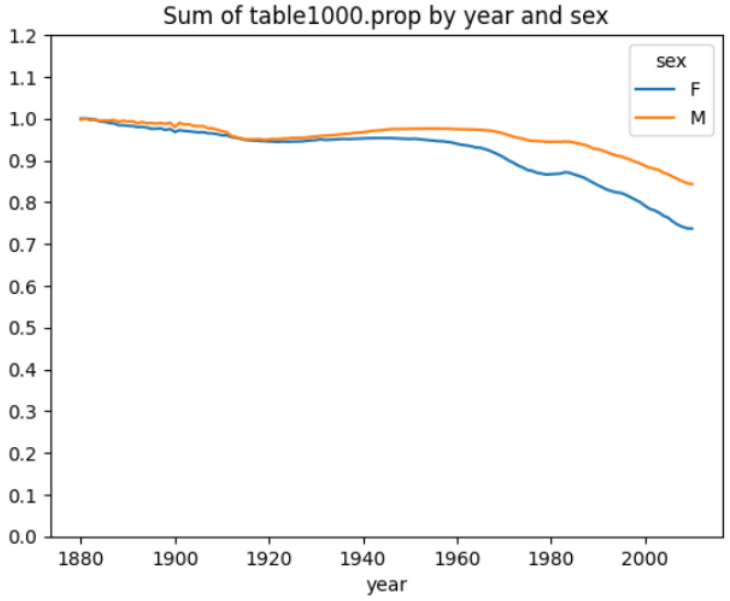
Tuy nhiên, sự sụt giảm trong tỷ lệ của các tên riêng lẻ không chỉ phản ánh sự thay đổi thị hiếu mà còn là kết quả của sự đa dạng hóa tên ngày càng tăng. Thay vì chọn một trong số ít tên phổ biến nhất, các bậc cha mẹ hiện đại có xu hướng chọn tên từ một kho tên rộng hơn. Để đo lường sự đa dạng này, ta có thể tính toán tỷ lệ tên (proportion) mà tổng số của chúng đạt đến 50% tổng số ca sinh trong mỗi năm. Tỷ lệ này được tính bằng cách: Nhóm dữ liệu top1000 theo năm và giới tính (year, sex; Tính tổng lũy tiến (cumsum) trên cột tỷ lệ (prop) đã được sắp xếp giảm dần; Tìm số lượng tên cần thiết để đạt đến ngưỡng 50% tổng số ca sinh.

Kết quả phân tích cho thấy, vào năm 1900, chỉ cần 70 tên nữ phổ biến nhất đã chiếm 50% tổng số ca sinh nữ. Trong khi đó, vào năm 2010, cần tới gần 2.000 tên để đạt được tỷ lệ tương tự. Điều này minh họa rõ ràng sự gia tăng đáng kể về tính đa dạng trong việc đặt tên trẻ sơ sinh.

3. Đo lường sự đa dạng tên

Một lời giải thích căn bản cho sự giảm tần suất của các tên phổ biến riêng lẻ là do ngày càng ít phụ huynh lựa chọn những cái tên thông thường cho con mình. Thay vào đó, họ có xu hướng tìm kiếm những cái tên ít phổ biến hơn, dẫn đến sự gia tăng đáng kể về tính đa dạng trong kho tên. Giả thuyết này được khám phá và xác nhận thông qua việc phân tích hai thước đo chính.

Đầu tiên, tổng tỷ lệ sinh được đại diện bởi 1.000 tên phổ biến nhất cho mỗi giới tính đã được tính toán.



Hình 3. 12 Biểu đồ tỷ lệ sinh của tổng 1000 tên hàng đầu

Kết quả cho thấy tổng tỷ lệ này giảm theo thời gian, phản ánh một xu hướng rõ ràng: những tên thuộc nhóm phổ biến nhất đang ngày càng chiếm một phần nhỏ hơn trong tổng số ca sinh.

Điều này trực tiếp chứng minh rằng tính đa dạng tên đang gia tăng, khi số lượng ca sinh bị phân tán trên một lượng lớn tên hơn.

	name	sex	births	year	prop
260877	Jacob	M	21875	2010	0.011523
260878	Ethan	M	17866	2010	0.009411
260879	Michael	M	17133	2010	0.009025
260880	Jayden	M	17030	2010	0.008971
260881	William	M	16870	2010	0.008887
...	...	...	...	...	...
261872	Camilo	M	194	2010	0.000102
261873	Destin	M	194	2010	0.000102
261874	Jaquan	M	194	2010	0.000102
261875	Jaydan	M	194	2010	0.000102
261876	Maxton	M	193	2010	0.000102

1000 rows × 5 columns

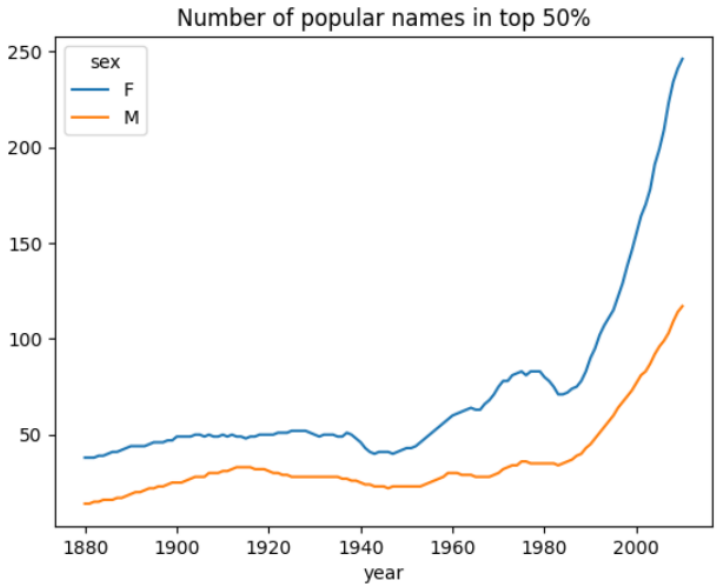
Hình 3. 13 Bảng các tên con trai từ năm 2010

Một thước đo mạnh mẽ khác để định lượng sự đa dạng là xác định số lượng tên riêng biệt (được sắp xếp theo mức độ phổ biến từ cao nhất đến thấp nhất) cần thiết để chiếm 50% tổng số ca sinh trong mỗi năm.

sex	F	M
year		
1880	38	14
1881	38	14
1882	38	15
1883	39	15
1884	39	16

Hình 3. 14 Số lượng Tên cần thiết để Chiếm 50% Tổng số Ca sinh theo các năm

Số lượng tên cần thiết để đạt ngưỡng 50% đã tăng lên đáng kể qua các năm, cho thấy sự phá vỡ các chuẩn mực đặt tên truyền thống. Vào năm 1900, chỉ cần 25 tên bé trai để chiếm 50% tổng số ca sinh nam. Ngược lại, vào năm 2010, con số này đã tăng lên 117 tên đối với bé trai.



Hình 3. 15 Biểu đồ sự da dạng tên của trẻ em qua từng năm

Phân tích cho thấy tên bé gái luôn đa dạng hơn tên bé trai qua tất cả các năm được ghi nhận. Hơn nữa, sự đa dạng này đối với cả hai giới tính chỉ ngày càng tăng lên theo thời gian, với tên nữ vẫn duy trì mức độ phân tán cao hơn.

4. Cuộc cách mạng "Chữ cái Cuối cùng"

Năm 2007, nhà nghiên cứu tên trẻ sơ sinh Laura Wattenberg đã chỉ ra rằng sự phân bố tên bé trai theo chữ cái cuối cùng đã thay đổi đáng kể trong 100 năm qua. Phân tích này khám phá các mô hình ngôn ngữ và âm vị học thay đổi trong việc đặt tên.

sex	F			M		
year	1910	1960	2010	1910	1960	2010
last_letter						
a	108376.0	691247.0	670605.0	977.0	5204.0	28438.0
b	NaN	694.0	450.0	411.0	3912.0	38859.0
c	5.0	49.0	946.0	482.0	15476.0	23125.0
d	6750.0	3729.0	2607.0	22111.0	262112.0	44398.0
e	133569.0	435013.0	313833.0	28655.0	178823.0	129012.0

Hình 3. 16 Bảng dữ liệu về sự phân bố tên trẻ em theo chữ cái cuối

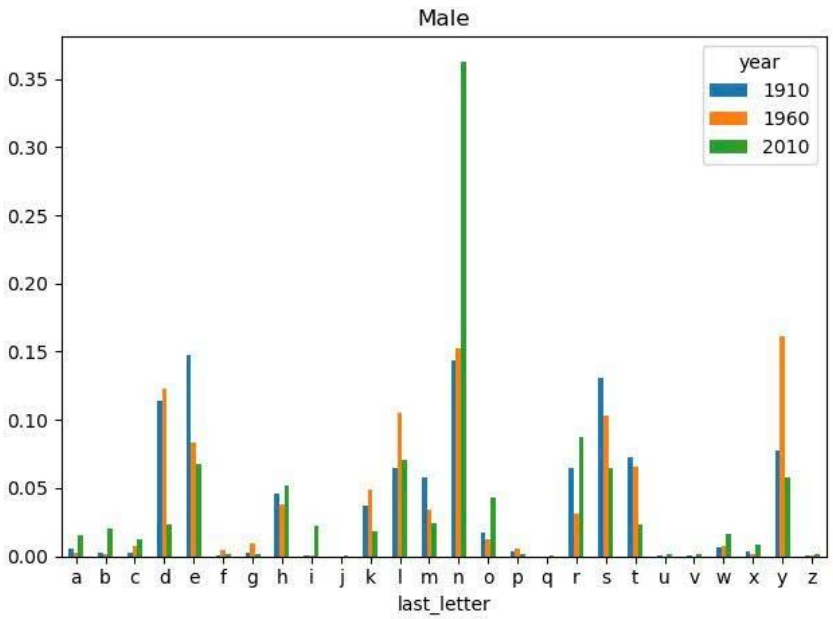
Để kiểm tra xu hướng này, ta thực hiện các bước sau trên tập dữ liệu top1000: Tạo một cột mới chứa chữ cái cuối cùng của mỗi tên; Nhóm dữ liệu

theo năm, giới tính và chữ cái cuối, sau đó tính tổng tỷ lệ (prop) để xác định tần suất tương đối của các tên kết thúc bằng mỗi chữ cái;

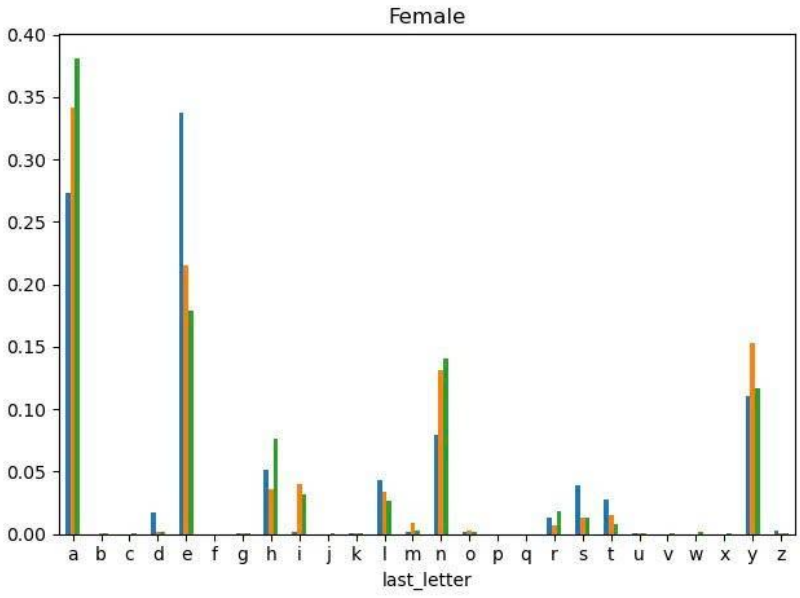
sex	F			M		
year	1910	1960	2010	1910	1960	2010
last_letter						
a	0.273390	0.341853	0.381240	0.005031	0.002440	0.014980
b	NaN	0.000343	0.000256	0.002116	0.001834	0.020470
c	0.000013	0.000024	0.000538	0.002482	0.007257	0.012181
d	0.017028	0.001844	0.001482	0.113858	0.122908	0.023387
e	0.336941	0.215133	0.178415	0.147556	0.083853	0.067959
f	NaN	0.000010	0.000055	0.000783	0.004325	0.001188
g	0.000144	0.000157	0.000374	0.002250	0.009488	0.001404
h	0.051529	0.036224	0.075852	0.045562	0.037907	0.051670
i	0.001526	0.039965	0.031734	0.000844	0.000603	0.022628
j	NaN	NaN	0.000090	NaN	NaN	0.000769
k	0.000121	0.000156	0.000356	0.036581	0.049384	0.018541
l	0.043189	0.033867	0.026356	0.065016	0.104904	0.070367
m	0.001201	0.008613	0.002588	0.058044	0.033827	0.024657

Hình 3. 17 Tỷ lệ tổng số ca sinh cho mỗi giới tính

Lọc dữ liệu cho tên bé trai và trực quan hóa xu hướng của các chữ cái cuối phổ biến.



Hình 3. 18 Biểu đồ sự phân hóa tên bé trai kết thúc bằng chữ cái

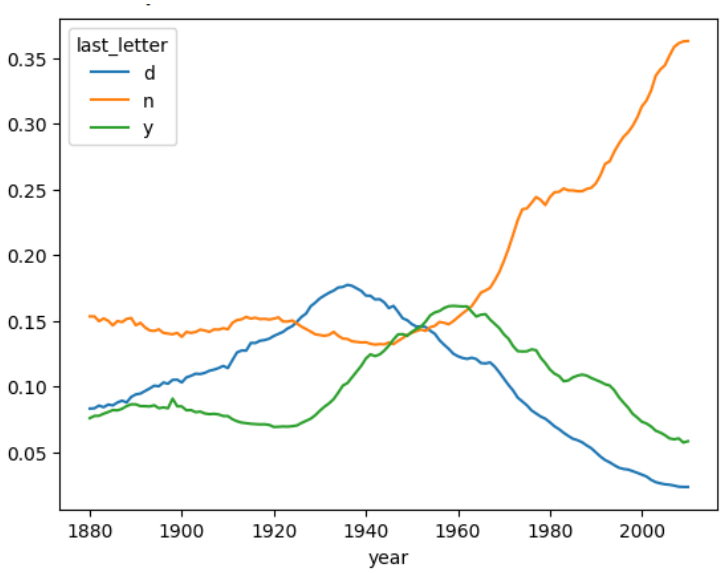


Hình 3. 19 Biểu đồ sự phân hóa tên bé gái kết thúc bằng chữ cái

Phân tích chữ cái cuối cùng của tên bé gái cho thấy sự thay đổi rõ rệt trong sở thích âm thanh: Tên kết thúc bằng 'a': Cho thấy một xu hướng tăng ổn định về mức độ phổ biến, khẳng định sự gia tăng ưu chuộng các tên con gái kết thúc bằng nguyên âm này (ví dụ: Isabella, Olivia). Tỉ lệ tăng từ khoảng 27% vào năm 1910 lên 38% vào năm 2010. Tên kết thúc bằng 'e': Có xu hướng giảm đáng kể. Tỉ lệ giảm từ khoảng 33.7% năm 1910 xuống còn 17.8% năm 2010. Tên kết thúc bằng 'y': Đạt mức phổ biến nhất vào năm 1960 (khoảng 15%) và sau đó giảm nhẹ vào năm 2010 (khoảng 11.6%). Tên kết thúc bằng 'h' và 'n': Cả hai đều cho thấy xu hướng tăng lên vào năm 2010 so với các năm trước đó.

Phân tích cho tên bé trai đã xác nhận phát hiện chính mà nhà nghiên cứu Laura Wattenberg đã chỉ ra, đó là sự thống trị của các tên kết thúc bằng 'n'. Tên kết thúc bằng 'n': Cho thấy xu hướng tăng trưởng mạnh mẽ nhất. Tỉ lệ tăng từ 14.3% năm 1910 lên 15.2% năm 1960 và sau đó tăng vọt lên 36.2% vào năm 2010, chiếm hơn một phần ba tổng số ca sinh bé trai trong năm đó. Tên kết thúc bằng 'd': Tỉ lệ giảm rõ rệt từ khoảng 11.4% năm 1910 (đạt mức cao nhất vào năm 1960: 12.3%) xuống chỉ còn 2.3% vào năm 2010. Tên kết thúc

bằng 'y': Giống như tên nữ, đạt mức phổ biến nhất vào năm 1960 (khoảng 16.1%) nhưng sau đó giảm mạnh vào năm 2010 (khoảng 5.8%).



Hình 3. 20 Biểu đồ thể hiện xu hướng các chữ cái cuối phổ biến

Xu hướng này trái ngược với sự sụt giảm của các tên kết thúc bằng các chữ cái khác như 'd', 'r', hoặc 'l' trong cùng kỳ.

5. Xu hướng Đổi Giới tính của Tên

Một xu hướng thú vị khác trong lịch sử đặt tên ở Hoa Kỳ là sự xuất hiện của những cái tên ban đầu phổ biến hơn với một giới tính nhưng sau đó lại được ưa chuộng hơn cho giới tính kia theo thời gian. Hiện tượng này thể hiện sự linh hoạt và thay đổi trong chuẩn mực giới tính gắn liền với các tên gọi.

Để minh họa cho sự thay đổi này, một ví dụ về nhóm tên "Lesley-like" (bao gồm Leslie, Lesley, Leslee, Lesli, Lesly) đã được kiểm tra: Lọc tập dữ liệu để chỉ chứa những tên thuộc nhóm "Lesley-like".

```
name
Leslee      1082
Lesley     35022
Lesli        929
Leslie    370429
Lesly      10067
Name: births, dtype: int64
```

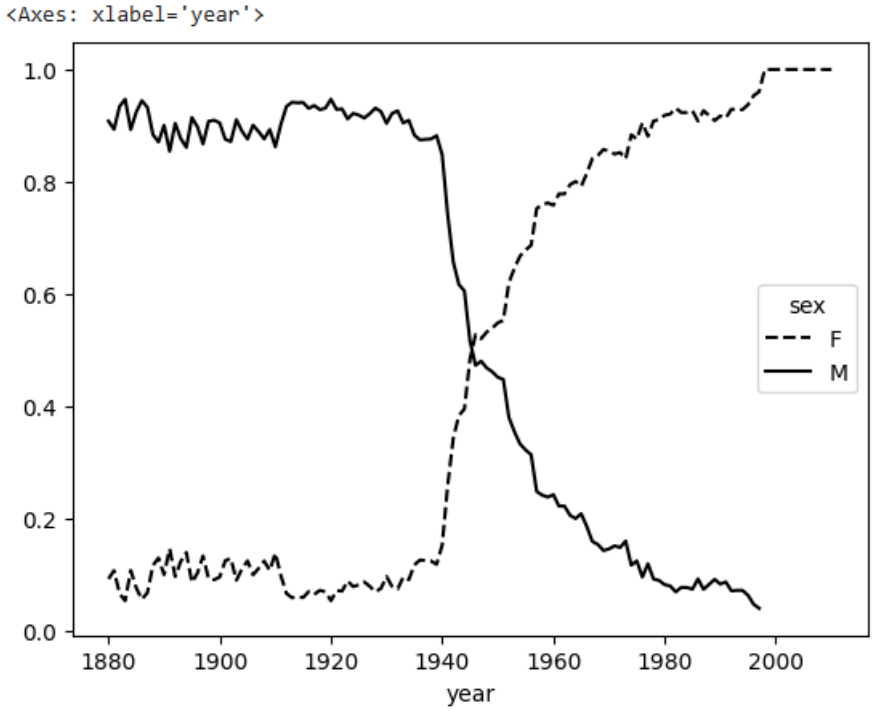
Hình 3. 21 Tập dữ liệu cho các tên bắt đầu bằng "Lesl"

Tính tổng tỷ lệ (prop) của các tên này theo năm và giới tính; Vẽ biểu đồ thể hiện tổng tỷ lệ của nhóm tên này, được phân tách theo giới tính (F và M).

sex	F	M
year		
2006	1.0	NaN
2007	1.0	NaN
2008	1.0	NaN
2009	1.0	NaN
2010	1.0	NaN

Hình 3. 22 Sự chuyển đổi trong mức độ ưa chuộng giới tính

Phân tích đã cho thấy một sự thay đổi rõ ràng: Nhóm tên "Lesl" đã chuyển đổi dần từ phổ biến hơn với bé trai sang phổ biến hơn với bé gái trong suốt thế kỷ 20.



Hình 3. 23 Biểu đồ phân tích sự phân chia giới tính theo nhóm tên "Lesl"

Biểu đồ thu được thể hiện sự chuyển dịch giới tính đáng kể trong việc sử dụng tên, phản ánh sự chấp nhận và ưa chuộng ngày càng tăng của những tên này đối với khán giả nữ.

Chương 5: Phân tích Cơ sở dữ liệu thực phẩm USDA

1. Tổng quan và chuẩn bị dữ liệu

Sử dụng cơ sở dữ liệu thông tin dinh dưỡng trong thực phẩm do Bộ Nông nghiệp Hoa Kỳ (USDA) cung cấp. Lập trình viên Ashley Williams đã tạo ra một phiên bản của cơ sở dữ liệu này ở định dạng JSON. Mỗi loại thực phẩm đều có một số đặc điểm nhận dạng cùng với hai danh sách chất dinh dưỡng và khẩu phần ăn. Dữ liệu ở dạng này không thực sự dễ phân tích, vì vậy chúng ta cần cố gắng sắp xếp dữ liệu thành một dạng tốt hơn.

Trong bài phân tích này, ta sẽ sử dụng module được tích hợp sẵn của Python là *json* để tải cơ sở dữ liệu lên, có thể thay thế bằng các thư viện khác. Cơ sở dữ liệu bao gồm 6636 phần tử, mỗi phần tử trong cơ sở dữ liệu là một từ điển chứa tất cả dữ liệu cho một loại thực phẩm. Trường "dinh dưỡng" là danh sách các từ điển, mỗi từ điển cho một chất dinh dưỡng, bao gồm các trường: *id, description, tags, manufacturer, group, portions, nutrients*.

```
{'value': 25.18,
 'units': 'g',
 'description': 'Protein',
 'group': 'Composition'}
```

Hình 4. 1 Thông tin thành phần dinh dưỡng đầu tiên của thực phẩm đầu tiên

Tổng quan danh sách thành phần dinh dưỡng của thực phẩm đầu tiên trong cơ sở dữ liệu có 162 dòng và 4 cột:

	value	units	description	group
0	25.180	g	Protein	Composition
1	29.200	g	Total lipid (fat)	Composition
2	3.060	g	Carbohydrate, by difference	Composition
3	3.280	g	Ash	Other
4	376.000	kcal	Energy	Energy
...	...	...	...	...
157	1.472	g	Serine	Amino Acids
158	93.000	mg	Cholesterol	Other
159	18.584	g	Fatty acids, total saturated	Other
160	8.275	g	Fatty acids, total monounsaturated	Other
161	0.830	g	Fatty acids, total polyunsaturated	Other

162 rows × 4 columns

Hình 4. 2 Bảng danh sách thành phần dinh dưỡng của thực phẩm đầu tiên

2. Xử lý dữ liệu

Trong bước này, ta sẽ xét trên hai DataFrame, một DataFrame con của dữ liệu gốc, đặt tên là info, lấy danh sách thông tin thực phẩm bao gồm tên thực phẩm, nhóm, ID và nhà sản xuất và tập nutrients, là một DataFrame được chuyển đổi từ mỗi danh sách thành phần dinh dưỡng của từng loại thực phẩm.

	description	group	id	manufacturer
0	Cheese, caraway	Dairy and Egg Products	1008	
1	Cheese, cheddar	Dairy and Egg Products	1009	
2	Cheese, edam	Dairy and Egg Products	1018	
3	Cheese, feta	Dairy and Egg Products	1019	
4	Cheese, mozzarella, part skim milk	Dairy and Egg Products	1028	

Hình 4. 3 Bảng thông tin của DataFrame info

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6636 entries, 0 to 6635
Data columns (total 4 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   description   6636 non-null   object
 1   group         6636 non-null   object
 2   id            6636 non-null   int64
 3   manufacturer  5195 non-null   object
dtypes: int64(1), object(3)
memory usage: 207.5+ KB
```

Hình 4. 4 Thông tin của DataFrame info

Từ kết quả cho thấy, *nhà sản xuất* (manufacturer) đang bị thiếu dữ liệu.

Xét sự phân bổ của các nhóm thực phẩm với value_counts, xem tần suất xuất hiện của mỗi giá trị duy nhất trong cột:

```
group
Vegetables and Vegetable Products    812
Beef Products                        618
Baked Products                       496
Breakfast Cereals                    403
Legumes and Legume Products          365
Fast Foods                           365
Lamb, Veal, and Game Products        345
Sweets                               341
Fruits and Fruit Juices              328
Pork Products                        328
Name: count, dtype: int64
```

Hình 4. 5 Kết quả của 10 giá trị đầu tiên về sự phân bổ của các nhóm thực phẩm

Bây giờ, để phân tích tất cả dữ liệu dinh dưỡng, cách dễ nhất là tập hợp các chất dinh dưỡng của từng loại thực phẩm vào một bảng lớn duy nhất. Thực hiện theo các bước:

Quy trình bắt đầu bằng việc khởi tạo một danh sách rỗng (nutrients) để lưu trữ các DataFrame tạm thời. Sau đó, mã thực hiện một vòng lặp qua từng bản ghi thực phẩm (rec) trong cơ sở dữ liệu gốc (db). Trong mỗi vòng lặp, nó truy cập danh sách các chất dinh dưỡng (rec["nutrients"]) và ngay lập tức chuyển danh sách từ điển này thành một DataFrame nhỏ (fnuts) bằng Pandas. Quan trọng nhất, mã lấy ID của thực phẩm (rec["id"]) và gán nó làm một cột mới cho tất cả các hàng trong DataFrame nhỏ này; bước này được gọi là gắn nhãn (tagging) và giúp truy vết nguồn gốc chất dinh dưỡng. Cuối cùng, các DataFrame nhỏ này được thêm vào danh sách nutrients. Sau khi vòng lặp kết thúc, hàm pd.concat được gọi để xếp chồng tất cả các DataFrame trong danh sách lên nhau theo chiều dọc, tạo ra một DataFrame lớn duy nhất. Kết quả sau khi thực hiện cho ra bao gồm 389355 dòng và 5 cột:

	value	units	description	group	id
0	25.180	g	Protein	Composition	1008
1	29.200	g	Total lipid (fat)	Composition	1008
2	3.060	g	Carbohydrate, by difference	Composition	1008
3	3.280	g	Ash	Other	1008
4	376.000	kcal	Energy	Energy	1008
...	...	...	...	...	...
389350	0.000	mcg	Vitamin B-12, added	Vitamins	43546
389351	0.000	mg	Cholesterol	Other	43546
389352	0.072	g	Fatty acids, total saturated	Other	43546
389353	0.028	g	Fatty acids, total monounsaturated	Other	43546
389354	0.041	g	Fatty acids, total polyunsaturated	Other	43546

389355 rows × 5 columns

Hình 4. 6 Thông tin DataFrame nutrients sau khi khởi tạo

Kết quả từ hàm duplicated().sum() cho ra số dữ liệu bị trùng là 14179 dòng. Tiến hành loại bỏ bằng drop_duplicates().

Ngoài việc trùng dữ liệu trong từng DataFrame, ta còn bị trùng tên cột "description" và "group" giữa 2 nguồn dữ liệu. Sử dụng hàm rename để đổi tên cột. Ở info sẽ đổi các cột: *description* thành *food*, *group* thành *fgroup*, ở nutrients sẽ đổi: *description* thành *nutrient*, *group* thành *nutgroup*. Kết quả sau khi đổi thu được như sau:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6636 entries, 0 to 6635
Data columns (total 4 columns):
 #   Column        Non-Null Count  Dtype 
---  ------        --------------  ----- 
 0   food          6636 non-null   object
 1   fgroup        6636 non-null   object
 2   id            6636 non-null   int64 
 3   manufacturer  5195 non-null   object
dtypes: int64(1), object(3)
memory usage: 207.5+ KB
```

Hình 4. 7 Thông tin của info sau khi đổi tên cột

```
<class 'pandas.core.frame.DataFrame'>
Index: 375176 entries, 0 to 389354
Data columns (total 5 columns):
 #   Column    Non-Null Count   Dtype
---  ------    --------------   -----
 0   value     375176 non-null  float64
 1   units     375176 non-null  object
 2   nutrient  375176 non-null  object
 3   nutgroup  375176 non-null  object
 4   id        375176 non-null  int64
dtypes: float64(1), int64(1), object(3)
memory usage: 17.2+ MB
```

Hình 4. 8 Thông tin của nutrients sau khi đổi tên cột

Chúng ta sẽ chạy hàm merge, truyền vào on là id. Kết quả cho ra được như sau:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 375176 entries, 0 to 375175
Data columns (total 8 columns):
 #   Column        Non-Null Count   Dtype
---  ------        --------------   -----
 0   value         375176 non-null  float64
 1   units         375176 non-null  object
 2   nutrient      375176 non-null  object
 3   nutgroup      375176 non-null  object
 4   id            375176 non-null  int64
 5   food          375176 non-null  object
 6   fgroup        375176 non-null  object
 7   manufacturer  293054 non-null  object
dtypes: float64(1), int64(1), object(6)
memory usage: 22.9+ MB
```

Hình 4. 9 DataFrame tổng hợp sau khi merge info và nutrients

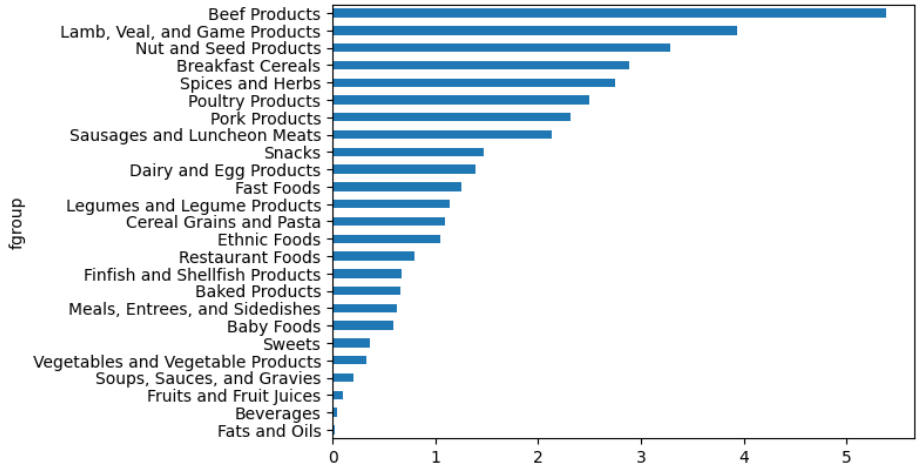
Kiểm tra ngẫu nhiên một dòng dữ liệu bằng iloc, ở đây là 30000, kết quả cho ra rất đầy đủ và chi tiết:

```
value                                              0.04
units                                                 g
nutrient                                        Glycine
nutgroup                                    Amino Acids
id                                                 6158
food          Soup, tomato bisque, canned, condensed
fgroup                     Soups, Sauces, and Gravies
manufacturer
Name: 30000, dtype: object
```

Hình 4. 10 Thông tin của dữ liệu thứ 30001

3. Phân tích và trực quan hóa dữ liệu

Quy trình phân tích và trực quan hóa dữ liệu gồm hai bước chính, sử dụng thư viện Pandas và Matplotlib. Đầu tiên, sử dụng hàm groupby để nhóm dữ liệu dinh dưỡng dựa trên sự kết hợp của loại chất dinh dưỡng (nutrient) và nhóm thực phẩm (fgroup). Sau khi nhóm, nó tính toán Trung vị (quantile(0.5)) của hàm lượng dinh dưỡng (value) cho từng nhóm. Điều này giúp xác định mức hàm lượng dinh dưỡng điển hình (Median) trong mỗi nhóm thực phẩm, ít bị ảnh hưởng bởi ngoại lai hơn so với Trung bình. Tiếp theo, trực quan hóa bằng cách lọc Series kết quả để chỉ lấy thông tin của Kẽm ("Zinc, Zn"), sắp xếp các giá trị Trung vị Kẽm đó theo thứ tự tăng dần và cuối cùng, tạo một biểu đồ thanh ngang để so sánh trực quan hàm lượng Kẽm điển hình giữa các nhóm thực phẩm khác nhau. Kết quả sau khi thực hiện thu được biểu đồ:



Hình 4. 11 Biểu đồ Thanh Ngang so sánh hàm lượng Kẽm (Zinc, Zn) Trung vị giữa các nhóm thực phẩm

Chú thích: Trục Y liệt kê các Nhóm Thực phẩm (fgroup), trong khi Trục X biểu thị Giá trị Trung vị của hàm lượng Kẽm tính bằng miligam (). Việc sắp xếp từ dưới lên trên cho phép dễ dàng so sánh các nhóm có hàm lượng thấp nhất đến cao nhất.

Biểu đồ cho thấy sự phân hóa rõ rệt về hàm lượng Kẽm. Ở đáy biểu đồ, các nhóm như Chất béo và Dầu (*Fats and Oils*), Đồ uống (*Beverages*), và Trái cây và Nước ép trái cây (*Fruits and Fruit Juices*) có hàm lượng Kẽm rất thấp (gần). Điều này phù hợp với hiểu biết dinh dưỡng rằng Kẽm chủ yếu không có trong các nhóm thực phẩm này. Ở mức trung bình, các nhóm như Thức ăn nhanh (*Fast Foods*), Sản phẩm từ Sữa và Trứng (*Dairy and Egg Products*), và Đồ ăn nhẹ (*Snacks*) có hàm lượng Kẽm vừa phải, dao động khoảng. Ngược lại, các nguồn cung cấp Kẽm mạnh nhất được thấy ở đỉnh biểu đồ. Sản phẩm từ Thịt bò (*Beef Products*) nổi bật với hàm lượng Kẽm cao nhất, vượt quá . Các nhóm như Thịt cừu, bê và thịt thú săn (*Lamb, Veal, and Game Products*) và Sản phẩm từ Hạt (*Nut and Seed Products*) cũng là những nguồn cung cấp Kẽm điển hình rất dồi dào, đạt mức khoảng trở lên.

Tóm lại, biểu đồ này khẳng định trực quan rằng thịt đỏ và các loại hạt là nguồn cung cấp Kẽm điển hình mạnh nhất trong chế độ ăn uống, trong khi các nhóm như chất béo, trái cây và đồ uống là những nguồn cung cấp rất ít khoáng chất này.

Tiếp theo, ta sử dụng phương pháp idxmax hoặc argmax Series để tìm ra loại thực phẩm nào có mật độ cao nhất trong mỗi chất dinh dưỡng:

Quy trình phân tích phức tạp nhằm xác định mẫu thực phẩm có hàm lượng dinh dưỡng cao nhất cho từng chất dinh dưỡng trong mỗi nhóm dinh dưỡng lớn. Quá trình bắt đầu bằng việc nhóm DataFrame ndata theo hai tiêu chí: Nhóm dinh dưỡng (nutgroup) và Tên chất dinh dưỡng (nutrient). Sau đó, mã định nghĩa một hàm (get_maximum) sử dụng idxmax() để tìm ra chỉ mục (index) của giá trị lớn nhất trong cột "value" (hàm lượng) bên trong mỗi nhóm. Hàm này sau đó được áp dụng (apply) cho tất cả các nhóm, trả về toàn bộ hàng của mẫu thực phẩm đạt giá trị tối đa đó, được lưu vào biến max_foods và lọc để chỉ giữ lại cột "value" và "food". Cuối cùng, cột "food" được xử lý để cắt ngắn tên thực phẩm xuống còn 50 ký tự, giúp làm sạch dữ liệu và dễ dàng hiển thị kết quả thống kê các giá trị cực đại này.

Vì DataFrame kết quả hơi lớn để hiển thị trong sách, nên đây chỉ là nhóm dinh dưỡng "Axit amin":

```
nutrient
Alanine                            Gelatins, dry powder, unsweetened
Arginine                                Seeds, sesame flour, low-fat
Aspartic acid                                     Soy protein isolate
Cystine               Seeds, cottonseed flour, low fat (glandless)
Glutamic acid                                     Soy protein isolate
Glycine                            Gelatins, dry powder, unsweetened
Histidine                Whale, beluga, meat, dried (Alaska Native)
Hydroxyproline    KENTUCKY FRIED CHICKEN, Fried Chicken, ORIGINA...
Isoleucine        Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Leucine           Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Lysine            Seal, bearded (Oogruk), meat, dried (Alaska Na...
Methionine                     Fish, cod, Atlantic, dried and salted
Phenylalanine     Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Proline                            Gelatins, dry powder, unsweetened
Serine            Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Threonine         Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Tryptophan         Sea lion, Steller, meat with fat (Alaska Native)
Tyrosine          Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Valine            Soy protein isolate, PROTEIN TECHNOLOGIES INTE...
Name: food, dtype: object
```

Hình 4. 12 Kết quả thống kê các giá trị cực đại của nhóm dinh dưỡng "Axit amin"

Bảng dữ liệu trên tiết lộ các nguồn protein cô đặc và các thực phẩm truyền thống của người Alaska Bản địa (thịt hải cẩu, cá voi, sư tử biển khô) là những thực phẩm có hàm lượng protein (amino acid) cực đại trong cơ sở dữ liệu USDA.

Chương 6: Phân tích Cơ sở dữ liệu Ủy ban Bầu cử Liên bang năm 2012

1. Tải Dữ Liệu Và Kiểm Tra Thông Tin Cơ Bản

Tải file CSV chứa dữ liệu đóng góp bầu cử năm 2012. Dữ liệu bao gồm thông tin như tên ứng cử viên, người đóng góp, số tiền, nghề nghiệp, v.v. Chúng ta sử dụng pd.read_csv để tải và info() để xem cấu trúc DataFrame.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1001731 entries, 0 to 1001730
Data columns (total 16 columns):
 #   Column             Non-Null Count    Dtype
---  ------             --------------    -----
 0   cmte_id            1001731 non-null  object
 1   cand_id            1001731 non-null  object
 2   cand_nm            1001731 non-null  object
 3   contbr_nm          1001731 non-null  object
 4   contbr_city        1001712 non-null  object
 5   contbr_st          1001727 non-null  object
 6   contbr_zip         1001620 non-null  object
 7   contbr_employer    988002 non-null   object
 8   contbr_occupation  993301 non-null   object
 9   contb_receipt_amt  1001731 non-null  float64
 10  contb_receipt_dt   1001731 non-null  object
 11  receipt_desc       14166 non-null    object
 12  memo_cd            92482 non-null    object
 13  memo_text          97770 non-null    object
 14  form_tp            1001731 non-null  object
 15  file_num           1001731 non-null  int64
dtypes: float64(1), int64(1), object(14)
memory usage: 122.3+ MB
```

Hình 5. 1 Thông tin tổng quát về DataFrame

Bộ dữ liệu được sử dụng trong nghiên cứu này gồm 1.001.731 dòng và 16 cột, phản ánh một lượng thông tin khá lớn và đa dạng. Dữ liệu được lưu trữ dưới nhiều kiểu khác nhau, bao gồm: một cột kiểu số thực (float64) mang tên contb_receipt_amt thể hiện giá trị các khoản đóng góp; một cột kiểu số nguyên (int64) là file_num, thường được dùng làm mã định danh hoặc chỉ số tệp; và mười bốn cột kiểu chuỗi (object), chứa các thông tin dạng văn bản như tên, địa chỉ hay mô tả chi tiết.

Tổng dung lượng của toàn bộ dữ liệu vào khoảng 122 MB, cho thấy đây là một tập dữ liệu có quy mô tương đối lớn, đòi hỏi khả năng xử lý và quản lý bộ nhớ phù hợp trong quá trình phân tích.

Đáng chú ý, một số cột trong bảng có chứa giá trị bị thiếu, tập trung chủ yếu ở các trường như employer, occupation, receipt_desc, memo_cd và memo_text. Việc tồn tại dữ liệu khuyết này có thể ảnh hưởng đến độ chính xác của kết quả phân tích, do đó cần được xem xét kỹ lưỡng và áp dụng các biện pháp xử lý thích hợp trong giai đoạn tiền xử lý dữ liệu.

#	Column name	Non-Null Count	Dtype	Giải thích
0	`cmte_id`	1,001,731	object	ID của ủy ban gây quỹ (committee ID)
1	`cand_id`	1,001,731	object	ID của ứng cử viên (candidate ID)
2	`cand_nm`	1,001,731	object	Tên ứng cử viên
3	`contbr_nm`	1,001,731	object	Tên người đóng góp (contributor)
4	`contbr_city`	1,001,712	object	Thành phố của người đóng góp *(19 giá trị bị thiếu)*
5	`contbr_st`	1,001,727	object	Bang (State) *(4 giá trị bị thiếu)*
6	`contbr_zip`	1,001,620	object	Mã ZIP *(111 giá trị bị thiếu)*
7	`contbr_employer`	988,002	object	Tên công ty / đơn vị công tác *(hơn 13 nghìn bị thiếu)*
8	`contbr_occupation`	993,301	object	Nghề nghiệp *(~7 nghìn bị thiếu)*
9	`contb_receipt_amt`	1,001,731	float64	**Số tiền đóng góp** (cột quan trọng nhất)
10	`contb_receipt_dt`	1,001,731	object	Ngày đóng góp (chưa được chuyển sang kiểu `datetime`)
11	`receipt_desc`	14,166	object	Mô tả giao dịch (nhiều ô bị trống)
12	`memo_cd`	92,482	object	Mã ghi chú (ít dữ liệu)
13	`memo_text`	97,770	object	Nội dung ghi chú
14	`form_tp`	1,001,731	object	Loại biểu mẫu
15	`file_num`	1,001,731	int64	Mã số tệp khai báo

Hình 5. 2 Bảng giải thích dữ liệu

2. Truy suất một bảng ghi mẫu

Để xem một ví dụ cụ thể về dữ liệu, lấy một dòng từ DataFrame và hiển thị dưới dạng Series. Ở đây là dòng thứ 123456.

```
cmte_id                             C00431445
cand_id                             P80003338
cand_nm                         Obama, Barack
contbr_nm                         ELLMAN, IRA
contbr_city                             TEMPE
contbr_st                                  AZ
contbr_zip                          852816719
contbr_employer      ARIZONA STATE UNIVERSITY
contbr_occupation                   PROFESSOR
contb_receipt_amt                        50.0
contb_receipt_dt                     01-DEC-11
receipt_desc                               NaN
memo_cd                                    NaN
memo_text                                  NaN
form_tp                                  SA17A
file_num                                772372
Name: 123456, dtype: object
```

Hình 5. 3 Dữ liệu dòng thứ 123456 dưới dạng Series

Có thể thấy Lệnh fec.iloc[123456] được sử dụng để truy cập một dòng dữ liệu cụ thể trong bảng theo vị trí chỉ mục. Kết quả trả về là một Series hiển thị chi tiết thông tin của một khoản đóng góp, chẳng hạn như tên ứng cử viên, người đóng góp và số tiền đóng góp (ví dụ: 50 USD). Việc sử dụng lệnh này giúp kiểm tra dữ liệu thực tế cũng như xác nhận cấu trúc của bảng, chẳng hạn như cột cand_nm biểu thị tên ứng cử viên và contb_receipt_amt thể hiện giá trị khoản đóng góp.

3. Lấy danh sách Ứng cử viên duy nhất

Lấy danh sách các ứng cử viên duy nhất từ cột cand_nm dưới dạng mảng NumPy. Dùng để xác định tất cả ứng cử viên trong dữ liệu, hữu ích để ánh xạ đảng phái sau này. Toàn bộ mảng có dạng:

```
array(['Bachmann, Michelle', 'Romney, Mitt', 'Obama, Barack',
       "Roemer, Charles E. 'Buddy' III", 'Pawlenty, Timothy',
       'Johnson, Gary Earl', 'Paul, Ron', 'Santorum, Rick',
       'Cain, Herman', 'Gingrich, Newt', 'McCotter, Thaddeus G',
       'Huntsman, Jon', 'Perry, Rick'], dtype=object)
```

Hình 5. 4 Mảng danh sách ứng viên

4. Ánh xạ Đáng phái cho Ứng cử viên

Vì bộ dữ liệu ban đầu không chứa thông tin về đảng phái của ứng cử viên, nên ta tiến hành tạo một từ điển ánh xạ giữa tên ứng cử viên và đảng phái tương ứng. Sau đó, sử dụng hàm map() để áp dụng ánh xạ này cho toàn bộ cột cand_nm và thêm một cột mới có tên party vào DataFrame, thể hiện đảng phái của từng ứng cử viên. Cuối cùng, ta kiểm tra lại DataFrame sau khi xử lý để đảm bảo cột party đã được thêm và ánh xạ chính xác.

	cmte_id	cand_id	cand_nm	contbr_nm	contbr_city	contbr_st	contbr_zip	contbr_employer	contbr_occupation	contb_receipt_amt	contb_receipt_dt	receipt_desc	memo_cd	memo_text	form_tp	file_num	party
0	C00410118	P20002978	Bachmann, Michelle	HARVEY, WILLIAM	MOBILE	AL	366010290	RETIRED	RETIRED	250.0	20-JUN-11	NaN	NaN	NaN	SA17A	736166	Republican
1	C00410118	P20002978	Bachmann, Michelle	HARVEY, WILLIAM	MOBILE	AL	366010290	RETIRED	RETIRED	50.0	23-JUN-11	NaN	NaN	NaN	SA17A	736166	Republican
2	C00410118	P20002978	Bachmann, Michelle	SMITH, LANIER	LANETT	AL	368633403	INFORMATION REQUESTED	INFORMATION REQUESTED	250.0	05-JUL-11	NaN	NaN	NaN	SA17A	749073	Republican
3	C00410118	P20002978	Bachmann, Michelle	BLEVINS, DARONDA	PIGGOTT	AR	724548253	NONE	RETIRED	250.0	01-AUG-11	NaN	NaN	NaN	SA17A	749073	Republican
4	C00410118	P20002978	Bachmann, Michelle	WARDENBURG, HAROLD	HOT SPRINGS NATION	AR	719016467	NONE	RETIRED	300.0	20-JUN-11	NaN	NaN	NaN	SA17A	736166	Republican

Hình 5. 5 5 dòng đầu tiên DataFrame sau khi đã xử lý

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1001731 entries, 0 to 1001730
Data columns (total 17 columns):
 #   Column             Non-Null Count    Dtype  
---  ------             --------------    -----  
 0   cmte_id            1001731 non-null  object 
 1   cand_id            1001731 non-null  object 
 2   cand_nm            1001731 non-null  object 
 3   contbr_nm          1001731 non-null  object 
 4   contbr_city        1001712 non-null  object 
 5   contbr_st          1001727 non-null  object 
 6   contbr_zip         1001620 non-null  object 
 7   contbr_employer    988002 non-null   object 
 8   contbr_occupation  993301 non-null   object 
 9   contb_receipt_amt  1001731 non-null  float64
 10  contb_receipt_dt   1001731 non-null  object 
 11  receipt_desc       14166 non-null    object 
 12  memo_cd            92482 non-null    object 
 13  memo_text          97770 non-null    object 
 14  form_tp            1001731 non-null  object 
 15  file_num           1001731 non-null  int64  
 16  party              1001731 non-null  object 
dtypes: float64(1), int64(1), object(15)
memory usage: 129.9+ MB
```

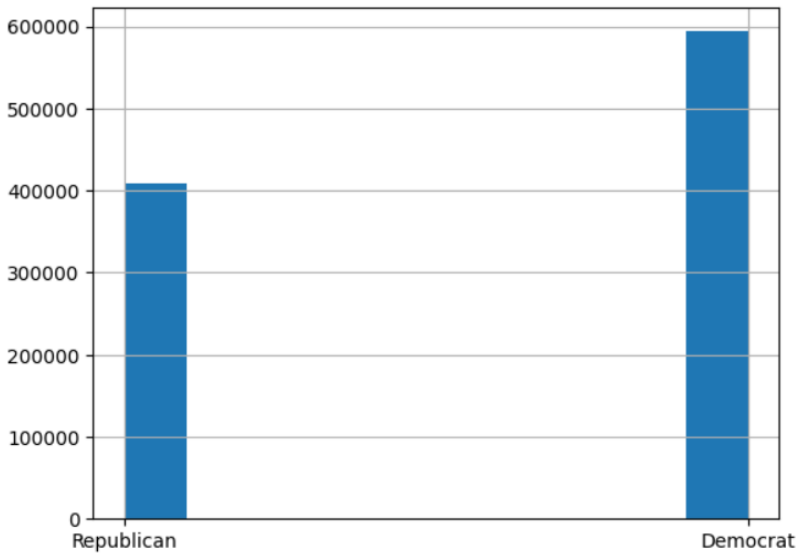
Hình 5. 6 Thông tin DataFrame sau khi thêm cột mới

Đếm số lượng giá trị trong cột 'party' (Democrat và Republican) bằng value_counts():

```
party
Democrat      593746
Republican    407985
Name: count, dtype: int64
```

Hình 5. 7 Số lượng giá trị trong cột party

Kiểm tra giá trị thiếu bằng isnull().sum(), kết quả cho ra không có giá trị nào thiếu.



Hình 5. 8 Biểu đồ histogram cho cột 'party'

Ta tạo một từ điển parties với khóa là tên ứng cử viên và giá trị là đảng phái (ví dụ giả định Gary Johnson được gán là Republican dù sau này có chuyển đảng). Khi kiểm tra ta có thể lấy một lát cắt của cột tên ứng cử viên như fec["cand_nm"][123456:123461] và áp dụng .map(parties) — phương thức map sẽ thay thế từng tên bằng giá trị đảng phái tương ứng từ từ điển, nếu không khớp sẽ trả về NaN (trong ví dụ này tất cả đều khớp). Sau đó thêm kết quả vào DataFrame bằng fec["party"] = fec["cand_nm"].map(parties). Cuối cùng dùng fec["party"].value_counts() để đếm tần suất từng đảng phái (ví dụ: số bản ghi của Democrat có thể nhiều hơn Republican). Ý nghĩa của bước này là bổ sung thông tin đảng phái cho mỗi khoản đóng góp, cho phép phân tích theo nhóm đảng trong các bước phân tích tiếp theo.

5. Lọc dữ liệu đóng góp dương

Dữ liệu có cả đóng góp và hoàn tiền (âm). Chúng ta lọc chỉ giữ đóng góp dương và tập trung vào Obama & Romney.

```
contb_receipt_amt
True     991475
False     10256
Name: count, dtype: int64
```

Hình 5. 9 Kết quả kiểm tra contributions âm/dương

```
Số lượng giá trị thiếu ở cột đóng góp: 0
cmte_id                   0
cand_id                   0
cand_nm                   0
contbr_nm                 0
contbr_city              19
contbr_st                 4
contbr_zip              111
contbr_employer       13729
contbr_occupation      8430
contb_receipt_amt         0
contb_receipt_dt          0
receipt_desc         987565
memo_cd              909249
memo_text            903961
form_tp                   0
file_num                  0
party                     0
dtype: int64
```

Hình 5. 10 Kết quả kiểm tra số lượng giá trị thiếu ở mỗi cột

Lọc lại DataFrame để chỉ giữ lại các dòng có giá trị đóng góp dương. Sau khi thực hiện bước lọc các đóng góp các giá trị dương từ Dataframe ban đầu là 1001731 dòng thành 991475 dòng => đã loại bỏ 10256 các dòng có đóng góp âm (hoàn tiền) hoặc bằng 0.

```
<class 'pandas.core.frame.DataFrame'>
Index: 991475 entries, 0 to 1001730
Data columns (total 17 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   cmte_id            991475 non-null  object
 1   cand_id            991475 non-null  object
 2   cand_nm            991475 non-null  object
 3   contbr_nm          991475 non-null  object
 4   contbr_city        991457 non-null  object
 5   contbr_st          991471 non-null  object
 6   contbr_zip         991373 non-null  object
 7   contbr_employer    983833 non-null  object
 8   contbr_occupation  989133 non-null  object
 9   contb_receipt_amt  991475 non-null  float64
 10  contb_receipt_dt   991475 non-null  object
 11  receipt_desc       5219 non-null    object
 12  memo_cd            89461 non-null   object
 13  memo_text          94274 non-null   object
 14  form_tp            991475 non-null  object
 15  file_num           991475 non-null  int64
 16  party              991475 non-null  object
dtypes: float64(1), int64(1), object(15)
memory usage: 136.2+ MB
```

Hình 5. 11 Thông tin DataFrame sau khi lọc

Tạo một DataFrame mới chỉ chứa các dòng là ứng viên Obama hoặc Romney:

```
<class 'pandas.core.frame.DataFrame'>
Index: 694282 entries, 411 to 701385
Data columns (total 17 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   cmte_id            694282 non-null  object
 1   cand_id            694282 non-null  object
 2   cand_nm            694282 non-null  object
 3   contbr_nm          694282 non-null  object
 4   contbr_city        694275 non-null  object
 5   contbr_st          694278 non-null  object
 6   contbr_zip         694234 non-null  object
 7   contbr_employer    693607 non-null  object
 8   contbr_occupation  693524 non-null  object
 9   contb_receipt_amt  694282 non-null  float64
 10  contb_receipt_dt   694282 non-null  object
 11  receipt_desc       2345 non-null    object
 12  memo_cd            87387 non-null   object
 13  memo_text          90672 non-null   object
 14  form_tp            694282 non-null  object
 15  file_num           694282 non-null  int64
 16  party              694282 non-null  object
dtypes: float64(1), int64(1), object(15)
memory usage: 95.3+ MB
```

Hình 5. 12 Thông tin DataFrame mới chỉ chứa các dòng là ứng viên Obama hoặc Romney

	cmte_id	cand_id	cand_nm	contbr_nm	contbr_city	contbr_st	contbr_zip	contbr_employer	contbr_occupation	contb_receipt_amt	contb_receipt_dt	receipt_desc	memo_cd	memo_text	form_tp	file_num	party
411	C00431171	P80003353	Romney, Mitt	ELDERBAUM, WILLIAM	DPO	AA	340230183	US GOVERNMENT	FOREIGN SERVICE OFFICER	25.0	01-FEB-12	NaN	NaN	NaN	SA17A	780124	Republican
412	C00431171	P80003353	Romney, Mitt	ELDERBAUM, WILLIAM	DPO	AA	340230183	US GOVERNMENT	FOREIGN SERVICE OFFICER	110.0	01-FEB-12	NaN	NaN	NaN	SA17A	780124	Republican
413	C00431171	P80003353	Romney, Mitt	CARLSEN, RICHARD	APO	AE	091280020	DEFENSE INTELLIGENCE AGENCY	INTELLIGENCE ANALYST	250.0	13-APR-12	NaN	NaN	NaN	SA17A	785689	Republican
414	C00431171	P80003353	Romney, Mitt	DELUCA, PIERRE	APO	AE	091280005	CISCO	ENGINEER	30.0	21-AUG-11	NaN	NaN	NaN	SA17A	760261	Republican
415	C00431171	P80003353	Romney, Mitt	SARGENT, MICHAEL	APO	AE	090120092	RAYTHEON TECHNICAL SERVICES CORP	COMPUTER SYSTEMS ENGINEER	100.0	07-MAR-12	NaN	NaN	NaN	SA17A	780128	Republican

Hình 5. 13 5 dòng đầu tiên của DataFrame đã lọc

Lệnh (fec["contb_receipt_amt"] > 0).value_counts() được dùng để tạo một Series kiểu boolean, trong đó giá trị True biểu thị các khoản đóng góp dương và False cho các giá trị âm hoặc bằng 0. Sau đó, phương thức

value_counts() sẽ đếm số lượng giá trị True và False, giúp ta kiểm tra tỷ lệ giữa các khoản đóng góp hợp lệ và không hợp lệ. Tiếp theo, sử dụng fec = fec[fec["contb_receipt_amt"] > 0], ta lọc dữ liệu để chỉ giữ lại các khoản đóng góp có giá trị dương, đồng thời cập nhật lại DataFrame gốc.

Sau đó, lệnh fec["cand_nm"].isin(["Obama, Barack", "Romney, Mitt"]) kiểm tra xem tên ứng cử viên trong cột cand_nm có nằm trong danh sách hai ứng cử viên chính (Obama, Barack và Romney, Mitt) hay không. Kết quả trả về là một mảng boolean có cùng số dòng với DataFrame, trong đó True cho biết dòng đó thuộc về một trong hai ứng cử viên. Cuối cùng, khi đặt điều kiện này trong cú pháp fec[...], pandas sẽ lọc và giữ lại các hàng có giá trị True.

Các bước này giúp làm sạch và thu gọn dữ liệu, chỉ tập trung vào những khoản đóng góp hợp lệ và hai ứng cử viên chính, phục vụ cho việc phân tích sâu hơn.

6. Thống kê nghề nghiệp

```
contbr_occupation
RETIRED                                   233990
INFORMATION REQUESTED                      35107
ATTORNEY                                   34286
HOMEMAKER                                  29931
PHYSICIAN                                  23432
INFORMATION REQUESTED PER BEST EFFORTS     21138
ENGINEER                                   14334
TEACHER                                    13990
CONSULTANT                                 13273
PROFESSOR                                  12555
Name: count, dtype: int64
```

Hình 5. 14 Top 10 nghề nghiệp phổ biến nhất

Lệnh fec["contbr_occupation"].value_counts() được sử dụng để đếm số lần xuất hiện của từng giá trị trong cột nghề nghiệp (contbr_occupation). Kết quả trả về là một Series được sắp xếp theo thứ tự giảm dần, trong đó giá trị có tần suất xuất hiện cao nhất sẽ nằm ở đầu. Khi thêm [:10], ta chỉ lấy 10 nghề nghiệp xuất hiện nhiều nhất trong dữ liệu.

Ý nghĩa: Giúp xác định những nhóm nghề nghiệp đóng góp nhiều nhất cho các chiến dịch, chẳng hạn như nhóm RETIRED (nghỉ hưu) thường chiếm tỷ lệ lớn trong các khoản quyên góp.

7. Làm Sạch Dữ Liệu Nghề Nghiệp Và Nhà Tuyển Dụng

Chuẩn hóa các giá trị lặp hoặc tương tự bằng từ điển và hàm. Từ điển ánh xạ để chuẩn hóa nghề nghiệp:

```
contbr_occupation
RETIRED          233990
NOT PROVIDED      57151
ATTORNEY          34286
HOMEMAKER         29931
PHYSICIAN         23432
ENGINEER          14334
TEACHER           13990
CONSULTANT        13273
PROFESSOR         12555
NOT EMPLOYED       9828
Name: count, dtype: int64
```

Hình 5. 15 Tần suất nghề nghiệp sau khi làm sạch

```
contbr_employer
RETIRED                                   206675
SELF-EMPLOYED                              94505
NOT EMPLOYED                               45877
INFORMATION REQUESTED                      36135
SELF                                       24385
INFORMATION REQUESTED PER BEST EFFORTS     22260
NONE                                       19929
HOMEMAKER                                  18269
SELF EMPLOYED                               6274
REQUESTED                                   4233
Name: count, dtype: int64
```

Hình 5. 16 Tần suất nhà tuyển dụng trước khi làm sạch

```
contbr_employer
RETIRED          206675
SELF-EMPLOYED    125164
NOT PROVIDED      58396
NOT EMPLOYED      45877
NONE              19929
HOMEMAKER         18269
REQUESTED          4233
UNEMPLOYED         2514
US ARMY            1817
STUDENT            1786
Name: count, dtype: int64
```

Hình 5. 17 Tần suất nhà tuyển dụng sau khi làm sạch

	cmte_id	cand_id	cand_nm	contbr_nm	contbr_city	contbr_st	contbr_zip	contbr_employer	contbr_occupation	contb_receipt_amt	contb_receipt_dt	receipt_desc	memo_cd	memo_text	form_tp	file_num	party
0	C00410118	P20002978	Bachmann, Michelle	HARVEY, WILLIAM	MOBILE	AL	366010290	RETIRED	RETIRED	250.0	20-JUN-11	NaN	NaN	NaN	SA17A	736166	Republican
1	C00410118	P20002978	Bachmann, Michelle	HARVEY, WILLIAM	MOBILE	AL	366010290	RETIRED	RETIRED	50.0	23-JUN-11	NaN	NaN	NaN	SA17A	736166	Republican
2	C00410118	P20002978	Bachmann, Michelle	SMITH, LANIER	LANETT	AL	368633403	NOT PROVIDED	NOT PROVIDED	250.0	05-JUL-11	NaN	NaN	NaN	SA17A	749073	Republican
3	C00410118	P20002978	Bachmann, Michelle	BLEVINS, DARONDA	PIGGOTT	AR	724548253	NONE	RETIRED	250.0	01-AUG-11	NaN	NaN	NaN	SA17A	749073	Republican
4	C00410118	P20002978	Bachmann, Michelle	WARDENBURG, HAROLD	HOT SPRINGS NATION	AR	719016467	NONE	RETIRED	300.0	20-JUN-11	NaN	NaN	NaN	SA17A	736166	Republican

Hình 5. 18 5 dòng đầu tiên của DataFrame sau khi làm sạch

Ta tạo một từ điển occ_mapping để chuẩn hóa dữ liệu trong cột contbr_occupation, ví dụ thay thế các biến thể như "Information Requested" thành "NOT PROVIDED" và "C.E.O." thành "CEO". Tiếp theo, định nghĩa hàm get_occ(x), trong đó sử dụng dict.get(key, default) để tìm giá trị ánh xạ trong từ điển: nếu khóa (key) tồn tại thì trả về giá trị tương ứng, còn nếu không có thì giữ nguyên giá trị gốc x.

Khi gọi fec["contbr_occupation"].map(get_occ), hàm get_occ() sẽ được áp dụng lên từng phần tử trong cột nghề nghiệp, giúp kiểm tra và thay thế các giá trị cần chuẩn hóa. Cách làm tương tự cũng được áp dụng cho từ điển emp_mapping nhằm làm sạch dữ liệu trong cột nhà tuyển dụng (employer).

Ý nghĩa: Bước này giúp làm sạch và thống nhất dữ liệu văn bản, tránh trùng lặp giữa các biến thể cùng nghĩa, từ đó nâng cao độ chính xác khi phân tích và nhóm dữ liệu theo nghề nghiệp hoặc nhà tuyển dụng.

8. Pivot Table Theo Nghề Nghiệp Và Đảng

Tổng hợp số tiền đóng góp theo nghề nghiệp và đảng, lọc những nghề vượt 2 triệu USD. Tạo pivot table: index là nghề nghiệp, columns là đảng, giá trị là tổng tiền, aggfunc='sum':

```
<class 'pandas.core.frame.DataFrame'>
Index: 45064 entries,    MIXED-MEDIA ARTIST / STORYTELLER to ~
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Democrat    29165 non-null  float64
 1   Republican  21078 non-null  float64
dtypes: float64(2)
memory usage: 1.0+ MB
```

Hình 5. 19 Thông tin của pivot table

party	Democrat	Republican
contbr_occupation		
MIXED-MEDIA ARTIST / STORYTELLER	100.0	NaN
AREA VICE PRESIDENT	250.0	NaN
RESEARCH ASSOCIATE	100.0	NaN
TEACHER	500.0	NaN
THERAPIST	3900.0	NaN

Hình 5. 20 5 dòng đầu tiên của pivot table

Lệnh fec.pivot_table(...) được sử dụng để tạo bảng tổng hợp (pivot table) từ DataFrame fec. Trong đó, values="contb_receipt_amt" chỉ định cột chứa giá trị cần tổng hợp, tức là số tiền đóng góp. index="contbr_occupation" xác định nghề nghiệp sẽ được dùng làm hàng (index) trong bảng kết quả, còn columns="party" đặt đảng phái làm tiêu đề cột (ví dụ: Democrat, Republican).

Hàm tổng hợp aggfunc="sum" quy định rằng nếu có nhiều bản ghi trùng nhau về nghề nghiệp và đảng phái, các giá trị sẽ được cộng dồn lại. Nếu không chỉ định, pivot_table mặc định sử dụng trung bình (np.mean).

Kết quả thu được là một DataFrame dạng bảng rộng, trong đó: Index: biểu thị từng nghề nghiệp (occupation), Cột: biểu thị từng đảng phái (party), Ô (i, j): là tổng số tiền đóng góp từ nghề nghiệp i cho đảng j. Nếu một cặp (nghề nghiệp, đảng phái) không tồn tại, giá trị tại ô tương ứng sẽ là NaN.

Sau khi tạo bảng này, ta lọc các hàng có tổng số tiền đóng góp (theo hàng) lớn hơn 2 triệu, nhằm tập trung phân tích vào những nhóm nghề nghiệp có mức đóng góp đáng kể.

party	Democrat	Republican
contbr_occupation		
ATTORNEY	11141982.97	7477194.43
CEO	2074974.79	4211040.52
CONSULTANT	2459912.71	2544725.45
ENGINEER	951525.55	1818373.70
EXECUTIVE	1355161.05	4138850.09
HOMEMAKER	4248875.80	13634275.78
INVESTOR	884133.00	2431768.92
LAWYER	3160478.87	391224.32
MANAGER	762883.22	1444532.37
NOT PROVIDED	4866973.96	20565473.01
OWNER	1001567.36	2408286.92
PHYSICIAN	3735124.94	3594320.24
PRESIDENT	1878509.95	4720923.76
PROFESSOR	2165071.08	296702.73
REAL ESTATE	528902.09	1625902.25
RETIRED	25305116.38	23561244.49
SELF-EMPLOYED	672393.40	1640252.54

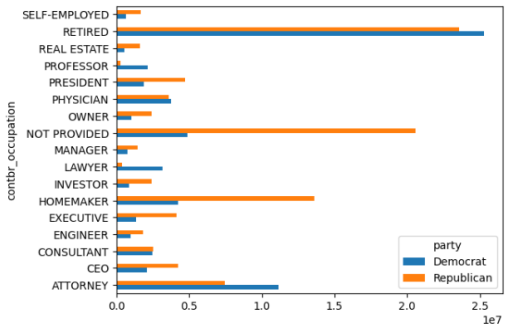
Hình 5. 21 Các hàng có tổng tiền (theo cột) > 2 triệu

Lệnh fec.pivot_table(...) tạo một bảng tổng hợp (pivot table) từ DataFrame, trong đó ta nhóm theo nghề nghiệp (index="contbr_occupation"), chia cột theo đảng phái (columns="party") và tổng hợp bằng tổng (aggfunc="sum") để thu được tổng tiền đóng góp của mỗi nghề cho từng đảng. Khi gọi by_occupation.sum(axis="columns"), ta tính tổng tiền đóng góp theo

mỗi hàng (tức tổng từ tất cả các đảng cho một nghề cụ thể). Việc so sánh lớn hơn 2000000 tạo ra một bộ lọc boolean để giữ lại những nghề có tổng đóng góp vượt quá 2.000.000, giúp ta tập trung vào những nhóm nghề đóng góp lớn và so sánh mức đóng góp giữa Democrat và Republican cho từng nghề.

9. Vẽ Biểu Đồ Theo Nghề Nghiệp

Vẽ biểu đồ ngang để trực quan hóa dữ liệu pivot.



Hình 5. 22 Biểu đồ ngang từ DataFrame over_2mm

Lệnh over_2mm.plot(kind="barh") được sử dụng để trực quan hóa dữ liệu dưới dạng biểu đồ cột ngang, trong đó Matplotlib sẽ tự động vẽ các thanh biểu diễn giá trị tổng đóng góp của hai đảng chính là Democrat và Republican theo từng nhóm nghề nghiệp. Việc sử dụng dạng biểu đồ ngang giúp hiển thị rõ ràng và dễ đọc hơn, đặc biệt khi các tên nghề nghiệp có độ dài khác nhau. Biểu đồ này mang ý nghĩa quan trọng trong việc so sánh trực quan mức độ đóng góp giữa các nhóm nghề và xu hướng chính trị của họ, từ đó cung cấp cái nhìn tổng quan về mối quan hệ giữa nghề nghiệp và hành vi tài trợ chính trị.

Quan sát biểu đồ thu được, có thể nhận thấy rằng nghề "RETIRED" (nghỉ hưu) là nhóm đóng góp nhiều nhất cho cả hai đảng, cho thấy sức ảnh hưởng lớn của người đã nghỉ hưu trong các chiến dịch tranh cử. Các nghề như "HOMEMAKER", "NOT PROVIDED", "ATTORNEY", "EXECUTIVE" và "CEO" cũng thể hiện mức đóng góp đáng kể, phản ánh khả năng tài chính cũng như mức độ tham gia của các nhóm nghề khác nhau vào hoạt động chính trị.

Đáng chú ý, các nghề như "ATTORNEY", "PROFESSOR" và "LAWYER" có xu hướng đóng góp nhiều hơn cho Đảng Dân chủ (Democrat), trong khi đó "CEO", "EXECUTIVE", "OWNER" và "PRESIDENT" lại nghiêng về phía Đảng Cộng hòa (Republican). Sự khác biệt này phản ánh xu hướng chính trị đặc trưng theo tầng lớp nghề nghiệp — những người thuộc nhóm quản lý cấp cao và chủ doanh nghiệp thường có khuynh hướng ủng hộ Đảng Cộng hòa, trong khi giới học giả, luật sư và giáo viên lại có xu hướng ủng hộ Đảng Dân chủ. Ngoài ra, nhóm "NOT PROVIDED" chiếm tỷ lệ đáng kể, cho thấy vẫn còn nhiều người không khai báo nghề nghiệp khi thực hiện

đóng góp, điều này có thể ảnh hưởng phần nào đến độ chính xác khi phân tích chi tiết theo ngành nghề.

10. Phân Loại Đóng Góp Theo Khoảng (Bucketing)

Trong bước này, ta tiến hành phân loại các khoản đóng góp tiền theo từng khoảng giá trị bằng cách sử dụng hàm pd.cut trong thư viện pandas. Trước hết, một mảng NumPy (np.array) được tạo ra để chứa các giá trị mốc (cut-off points), dùng làm giới hạn cho từng khoảng phân chia dữ liệu. Các mốc này được xác định dựa trên giá trị tiền đóng góp, ví dụ từ 0 đô trở lên, 0–1 đô, 1–10 đô, 10–100 đô, cho đến 1.000.000–10.000.000 đô.

Việc chia dữ liệu thành các khoảng như vậy giúp phân tầng mức độ đóng góp, qua đó dễ dàng hơn trong việc thống kê, so sánh và trực quan hóa. Nhờ đó, ta có thể nhận diện được xu hướng đóng góp theo từng mức tiền, phát hiện sự khác biệt giữa các nhóm người đóng góp nhỏ lẻ và những người quyên góp với số tiền lớn. Đây là bước khởi đầu quan trọng trong quá trình phân tích dữ liệu, giúp làm nổi bật đặc trưng phân bố của các khoản tài trợ trong toàn bộ tập dữ liệu.

Hàm pd.cut() của pandas được dùng để phân chia dữ liệu số liên tục (continuous data) thành các khoảng (bins). Ở đây, ta đang phân loại (binning) cột contb_receipt_amt (số tiền đóng góp) trong DataFrame fec_mrbo → theo các khoảng giá trị đã được định nghĩa trong biến bins.

Trong bước này, ta tiến hành phân loại (binning) các giá trị tiền đóng góp bằng cách sử dụng hàm pd.cut() của Pandas. Cụ thể, cột contb_receipt_amt trong bảng dữ liệu fec_mrbo chứa thông tin về số tiền đóng góp của hai ứng cử viên Obama và Romney (vì dữ liệu đã được lọc từ trước chỉ còn hai người này). Tiếp đó, ta tạo một mảng NumPy có tên bins chứa các ranh giới phân loại như [0, 1, 10, 100, 1000, 10000, 100000, 1000000, 10000000], tương ứng với các mức giá trị đóng góp khác nhau. Hàm pd.cut() sẽ duyệt qua từng giá trị trong cột contb_receipt_amt, xác định xem mỗi giá trị thuộc vào khoảng nào trong các mốc đã cho. Kết quả trả về là một Series có cùng độ dài với dữ liệu ban đầu, nhưng thay vì hiển thị giá trị số tiền cụ thể, mỗi dòng sẽ thể hiện khoảng giá trị (Interval) mà số tiền đó thuộc về. Việc phân loại này giúp dữ liệu trở nên dễ dàng phân tích, thống kê và trực quan hóa hơn, đặc biệt khi cần so sánh mức độ đóng góp giữa các nhóm người ủng hộ khác nhau.

```
411        (10, 100]
412       (100, 1000]
413       (100, 1000]
414        (10, 100]
415        (10, 100]
             ...
701381     (10, 100]
701382    (100, 1000]
701383       (1, 10]
701384     (10, 100]
701385    (100, 1000]
Name: contb_receipt_amt, Length: 694282, dtype: category
Categories (8, interval[int64, right]): [(0, 1] < (1, 10] < (10, 100] < (100, 1000] < (1000, 10000] < (10000, 100000] < (100000, 1000000] < (1000000, 10000000]]
```

Hình 5. 23 Label phân loại số tiền vào bins

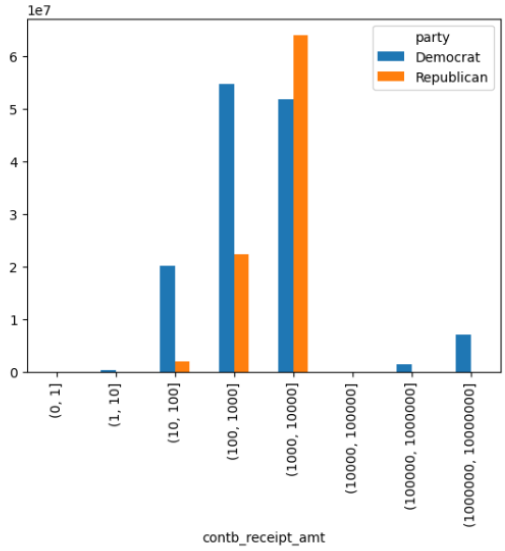
Lệnh np.array([...]) được sử dụng để tạo mảng chứa các giá trị ranh giới nhằm chia dữ liệu thành các khoảng như (0,1], (1,10], (10,100]...; sau đó, pd.cut(..., bins, right=True) sẽ dựa trên các ranh giới này để phân loại từng giá trị trong cột thành các khoảng tương ứng, trong đó right=True có nghĩa là bao gồm cả giới hạn bên phải của khoảng. Quá trình này giúp biến đổi dữ liệu số thành các nhóm (bins) có ý nghĩa, phục vụ cho việc thống kê, phân tích hoặc vẽ biểu đồ histogram, qua đó ta dễ dàng quan sát sự phân bố của các mức đóng góp theo từng khoảng tiền.

```
contb_receipt_amt
(10, 100]               404133
(100, 1000]             197348
(1000, 10000]            48470
(1, 10]                  43751
(0, 1]                     570
(1000000, 10000000]          4
(10000, 100000]              3
(100000, 1000000]            3
Name: count, dtype: int64
```

Hình 5. 24 Bảng thống kê số lượng người quyên góp trong mỗi khoảng tiền

party	Democrat	Republican
contb_receipt_amt		
(0, 1]	318.24	77.00
(1, 10]	337267.62	29819.66
(10, 100]	20288981.41	1987783.76
(100, 1000]	54798531.46	22363381.69
(1000, 10000]	51753705.67	63942145.42
(10000, 100000]	59100.00	12700.00
(100000, 1000000]	1490683.08	0.00
(1000000, 10000000]	7148839.76	0.00

Hình 5. 25 Bảng tổng số tiền theo từng khoảng và đảng phái



Hình 5. 26 Biểu đồ thanh từ thống kê lượng người quyên góp trong mỗi khoảng tiền của mỗi đảng

Biểu đồ thể hiện sự phân bố mức đóng góp của cử tri cho hai ứng cử viên Obama và Romney cho thấy phần lớn các khoản đóng góp tập trung ở các khoảng nhỏ, chủ yếu trong khoảng từ 10 đến 100 đô, 100 đến 1.000 đô và 1.000 đến 10.000 đô. Đáng chú ý, Obama nhận được tỷ lệ đóng góp cao hơn ở các mức thấp, đặc biệt là dưới 100 đô, phản ánh mạng lưới quyên góp rộng rãi và sự tham gia của đông đảo người ủng hộ với các khoản nhỏ. Ngược lại, Romney lại nhận được tỷ lệ đóng góp lớn hơn ở các mức cao, đặc biệt từ 1.000 đô trở lên, cho thấy chiến lược gây quỹ của ông phụ thuộc nhiều hơn vào những khoản quyên góp lớn từ số ít nhà tài trợ. Các khoản đóng góp cực lớn (trên 100.000 đô) hầu như chỉ xuất hiện ở phía Obama, cho thấy ông cũng nhận được sự ủng hộ đáng kể từ một số nhà tài trợ lớn. Nhìn chung, biểu đồ phản ánh rõ nét sự khác biệt trong cơ cấu và chiến lược gây quỹ giữa hai ứng viên trong chiến dịch tranh cử.

11. Nhóm Và Tính Toán Theo Khoảng Đóng Góp

Bước này nhóm dữ liệu theo ứng cử viên và các khoảng giá trị đóng góp, sau đó đếm số lượng trong từng nhóm để biết mỗi ứng cử viên nhận bao nhiêu đóng góp ở mỗi khoảng. Cuối cùng, dùng unstack() để chuyển kết quả thành bảng hai chiều, giúp so sánh trực quan giữa các ứng cử viên theo từng mức đóng góp.

cand_nm	Obama, Barack	Romney, Mitt
contb_receipt_amt		
(0, 1]	493	77
(1, 10]	40070	3681
(10, 100]	372280	31853
(100, 1000]	153991	43357
(1000, 10000]	22284	26186
(10000, 100000]	2	1
(100000, 1000000]	3	0
(1000000, 10000000]	4	0

Hình 5. 27 Bảng đã được unstack ra

Tính tổng tiền theo nhóm và unstack để thành bảng rồi hiển thị tổng tiền theo nhóm:

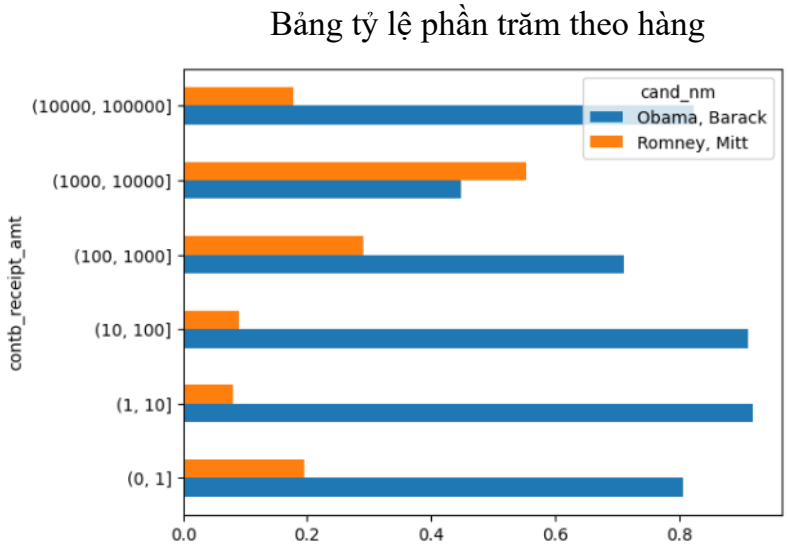
cand_nm	Obama, Barack	Romney, Mitt
contb_receipt_amt		
(0, 1]	318.24	77.00
(1, 10]	337267.62	29819.66
(10, 100]	20288981.41	1987783.76
(100, 1000]	54798531.46	22363381.69
(1000, 10000]	51753705.67	63942145.42
(10000, 100000]	59100.00	12700.00
(100000, 1000000]	1490683.08	0.00
(1000000, 10000000]	7148839.76	0.00

Hình 5. 28 Bảng tổng tiền theo nhóm

Bước này nhằm chuẩn hóa dữ liệu để dễ so sánh tỷ lệ giữa hai ứng cử viên trong từng khoảng tiền. Cụ thể, bucket_sums.sum(axis="columns") tính tổng đóng góp của từng khoảng (tức tổng theo hàng), sau đó .div(..., axis="index") chia từng giá trị trong hàng cho tổng đó. Kết quả là mỗi ô biểu thị tỷ lệ phần trăm đóng góp của mỗi ứng cử viên trong cùng một khoảng giá trị, giúp quan sát rõ ai chiếm ưu thế ở từng mức đóng góp.

cand_nm	Obama, Barack	Romney, Mitt
contb_receipt_amt		
(0, 1]	80.518166	19.481834
(1, 10]	91.876684	8.123316
(10, 100]	91.076874	8.923126
(100, 1000]	71.017590	28.982410
(1000, 10000]	44.732551	55.267449
(10000, 100000]	82.311978	17.688022
(100000, 1000000]	100.000000	0.000000
(1000000, 10000000]	100.000000	0.000000

Hình 5. 29 Bảng tỷ lệ phần trăm theo hàng



Hình 5. 30 Biểu đồ ngang biểu diễn tỷ lệ phần trăm theo hàng (loại bỏ 2 dòng cuối)

Lệnh fec_mrbo.groupby(["cand_nm", labels]) thực hiện phân nhóm theo hai cấp — trước tiên theo tên ứng cử viên, sau đó theo các khoảng giá trị (labels); gọi grouped.size() sẽ đếm số bản ghi trong mỗi nhóm (số khoản đóng góp), còn grouped["contb_receipt_amt"].sum() sẽ tính tổng tiền cho mỗi nhóm thay vì đếm. Phương thức .unstack(level=0) biến cấp cand_nm (level 0) thành các cột trong bảng kết quả, chuyển từ MultiIndex sang dạng bảng hai chiều thuận tiện cho so sánh. Sau khi có bucket_sums (tổng tiền theo từng khoảng và ứng cử viên), ta chuẩn hóa bằng cách .div(..., axis="index") — tức là chia từng ô cho tổng của hàng đó (tổng toàn bộ ứng cử viên trong cùng một khoảng) để thu được tỷ lệ phần trăm mỗi ứng cử viên đóng góp trong mỗi bin. Cuối cùng normed_sums[:-2].plot(kind="barh") vẽ biểu đồ cột ngang của các hàng đã chuẩn hóa, loại bỏ hai hàng cuối cùng (thường là các bin cực lớn/ít quan tâm) để biểu đồ dễ đọc hơn. Ý nghĩa của cả chuỗi thao tác là cho phép so sánh

trực quan và định lượng xem Obama và Romney chiếm bao nhiêu phần trăm trong từng mức đóng góp (từ nhỏ đến lớn), từ đó rút ra khác biệt về cơ cấu và chiến lược gây quỹ giữa hai ứng cử viên.

12. Thống kê theo Tiểu bang

Nhóm theo ứng cử viên và tiểu bang, sau đó đếm số lượng trong từng nhóm, unstack để thành bảng. Xử lý NaN bằng cách thay bằng 0, kết quả thu được như sau:

cand_nm	Obama, Barack	Romney, Mitt
contbr_st		
AA	56405.00	135.00
AB	2048.00	0.00
AE	42973.75	5680.00
AK	281840.15	86204.24
AL	543123.48	527303.51
...	...	...
WI	1130155.46	270316.32
WV	169154.47	126725.12
WY	194046.74	252595.84
XX	0.00	400250.00
ZZ	5963.00	0.00

67 rows × 2 columns

Hình 5. 31 Bảng tổng tiền theo tiểu bang và ứng cử viên

Lọc tiểu bang có tổng > 100.000:

cand_nm	Obama, Barack	Romney, Mitt
contbr_st		
AK	281840.15	86204.24
AL	543123.48	527303.51
AR	359247.28	105556.00
AZ	1506476.98	1888436.23
CA	23824984.24	11237636.60
CO	2132429.49	1506714.12
CT	2068291.26	3499475.45
DC	4373538.80	1025137.50
DE	336669.14	82712.00
FL	7318178.58	8338458.81

Hình 5. 32 Bảng danh sách tiểu bang có tổng > 100.000 (10 dòng đầu tiên)

Lệnh groupby(["cand_nm", "contbr_st"]) thực hiện việc nhóm dữ liệu theo hai tiêu chí: tên ứng cử viên (cand_nm) và tiểu bang đóng góp (contbr_st). Sau đó, .sum().unstack(level=0) sẽ tính tổng số tiền đóng góp cho mỗi cặp (ứng cử viên, tiểu bang) và chuyển cấp cand_nm (ứng cử viên) thành các cột riêng biệt, giúp dễ dàng so sánh giữa các ứng cử viên theo từng tiểu bang. Tiếp đến, .fillna(0) được dùng để thay thế các giá trị NaN (trường hợp tiểu bang không có đóng góp cho một ứng cử viên nào đó) bằng 0, đảm bảo dữ liệu nhất quán khi tính toán tiếp. Cuối cùng, totals.sum(axis="columns") > 100000 là

điều kiện lọc, chỉ giữ lại những tiểu bang có tổng đóng góp vượt 100.000 đô la, loại bỏ các tiểu bang có mức đóng góp thấp. Ý nghĩa của thao tác này là giúp ta xác định và tập trung phân tích các tiểu bang có mức quyên góp lớn, từ đó đánh giá mức độ ủng hộ của từng khu vực dành cho mỗi ứng cử viên trong chiến dịch tranh cử.

13. Tính Tỷ Lệ Phần Trăm Theo Tiểu Bang

Chia từng hàng cho tổng hàng để lấy % và sắp xếp theo cột Obama:

cand_nm contbr_st	Obama, Barack	Romney, Mitt
VT	0.946983	0.053017
ME	0.908825	0.091175
PR	0.881192	0.118808
HI	0.876774	0.123226
MN	0.855913	0.144087
NM	0.843590	0.156410
IL	0.819226	0.180774
DC	0.810113	0.189887
WI	0.806982	0.193018
DE	0.802776	0.197224

Hình 5. 33 Bảng Tỷ Lệ Phần Trăm Theo Tiểu Bang sắp xếp theo cột Obama

Hàm sort_values() được sử dụng để sắp xếp các hàng trong DataFrame dựa trên giá trị của một hoặc nhiều cột. Ở đây, tham số by="Obama, Barack" cho biết việc sắp xếp sẽ dựa trên cột thể hiện tổng số tiền đóng góp cho ứng cử viên Barack Obama. Tham số ascending=False giúp sắp xếp theo thứ tự giảm dần, nghĩa là các tiểu bang có mức đóng góp cao nhất cho Obama sẽ được hiển thị trước. Nếu đặt ascending=True, dữ liệu sẽ được sắp xếp ngược lại — từ các giá trị nhỏ nhất lên lớn nhất. Cách sắp xếp này giúp dễ dàng nhận biết những khu vực có mức ủng hộ tài chính lớn nhất cho từng ứng cử viên.

Sắp xếp theo cột Romney và in ra top 10 bang có tỷ lệ đóng góp cao nhất cho Romney:

cand_nm	Obama, Barack	Romney, Mitt
contbr_st		
XX	0.000000	1.000000
UT	0.122689	0.877311
ID	0.200608	0.799392
SD	0.341899	0.658101
LA	0.356026	0.643974
CT	0.371476	0.628524
MS	0.371535	0.628465
OK	0.414515	0.585485
TN	0.424589	0.575411
WY	0.434456	0.565544

Hình 5. 34 Bảng danh sách top 10 bang có tỷ lệ đóng góp cao nhất cho Romney

Câu lệnh totals.div(..., axis="index") được sử dụng để chuẩn hóa dữ liệu theo hàng, tức là tính tỷ lệ phần trăm đóng góp của mỗi ứng cử viên trong từng tiểu bang. Cụ thể, tổng đóng góp của mỗi bang được dùng làm mẫu số, sau đó mỗi giá trị trong hàng (tương ứng với một ứng cử viên) được chia cho tổng đó. Kết quả cho thấy, trong từng bang, ứng cử viên nào nhận được tỷ lệ đóng góp cao hơn, giúp so sánh trực quan mức độ ủng hộ giữa Obama và Romney theo khu vực.

Chương 7: Kết luận

1. Tổng hợp các kỹ thuật nổi bật trong 5 phần phân tích

Phần phân tích dữ liệu đã sử dụng năm bộ dữ liệu thực tế đa dạng - bao gồm dữ liệu Bitly, MovieLens 1M, Tên trẻ sơ sinh Hoa Kỳ, Cơ sở dữ liệu Thực phẩm USDA, và Dữ liệu Ủy ban Bầu cử Liên bang (FEC) 2012 - để minh họa một cách toàn diện các kỹ thuật xử lý và phân tích dữ liệu mạnh mẽ, cốt lõi là thư viện pandas và NumPy.

Quy trình bắt đầu bằng việc Tải và Kết hợp Dữ liệu từ các nguồn và định dạng khác nhau. Đối với dữ liệu web như Bitly, việc xử lý định dạng JSON được thực hiện bằng mô-đun json của Python. Các bộ dữ liệu phức tạp hơn như MovieLens 1M, được lưu trữ dưới dạng nhiều bảng riêng biệt (ratings, users, movies), đã được tải bằng pandas.read_table và sau đó hợp nhất thành một DataFrame thống nhất bằng lệnh pandas.merge dựa trên các khóa chung. Cuối cùng, đối với các dữ liệu lớn phân chia theo tệp (ví dụ: dữ liệu Tên trẻ sơ sinh hàng năm), kỹ thuật lặp qua các tệp và sử dụng pandas.concat đã được áp dụng để tổng hợp thành một DataFrame duy nhất.

Bước tiếp theo tập trung vào Làm sạch và Chuyển đổi Dữ liệu để đảm bảo chất lượng và tính nhất quán. Việc xử lý các giá trị thiếu (NaN) và không rõ ràng là rất quan trọng: chúng tôi sử dụng kiểm tra điều kiện để loại trừ các bản ghi thiếu trường dữ liệu quan trọng và áp dụng phương thức fillna hoặc Boolean array indexing để thay thế các giá trị NaN hoặc chuỗi rỗng ("") bằng các giá trị thay thế như "Missing" hoặc "Unknown". Về mặt chuyển đổi, các phương thức chuỗi của pandas (ví dụ: str.split) được sử dụng để trích xuất thông tin chi tiết từ các chuỗi phức tạp (như thông tin trình duyệt hoặc thể loại phim). Các cột phân loại mới (như cột hệ điều hành) được tạo dựa trên điều kiện logic sử dụng numpy.where. Để chuẩn hóa các mục nhập không nhất quán (ví dụ: nghề nghiệp FEC), map với một dictionary đã được sử dụng. Đặc biệt, phương thức explode đã được dùng để phân tách các bản ghi đa giá trị (ví dụ: danh sách thể loại phim) thành các hàng riêng biệt, sẵn sàng cho việc phân tích.

Phần cuối cùng tập trung vào Tổng hợp và Nhóm Dữ liệu để rút ra các insight có ý nghĩa. Việc đếm tần suất được thực hiện hiệu quả nhất bằng

phương thức value_counts() của pandas Series. Phân tích cốt lõi được thực hiện bằng groupby() để nhóm dữ liệu theo các thuộc tính (ví dụ: múi giờ, hệ điều hành) và áp dụng các hàm tổng hợp. Phương thức pivot_table được sử dụng để tính toán các thống kê tổng hợp và định hình lại dữ liệu. Các phép tính nâng cao hơn bao gồm việc sử dụng apply trong groupby để tính tỷ lệ phần trăm (ví dụ: tỷ lệ sinh của một tên so với tổng ca sinh trong năm), sử dụng nlargest để xác định các nhóm có tổng giá trị cao nhất, và tính toán sự bất đồng (ví dụ: sự khác biệt rating trung bình giữa các nhóm). Ngoài ra, hàm pandas.cut đã được áp dụng để chia các giá trị liên tục (ví dụ: số tiền quyên góp) thành các khoảng rời rạc phục vụ cho việc phân loại.

2. So sánh hiệu quả giữa các phương pháp

Tài liệu cung cấp sự so sánh rõ ràng về hiệu quả và sự tiện lợi giữa các phương pháp:

Nhiệm vụ	Phương pháp	Mức độ Hiệu quả/Tiện lợi
Đếm tần suất	Python thuần (sử dụng dictionary)	Cần nhiều mã hơn.
	collections.Counter	Đơn giản hơn Python thuần.
	pandas.Series.value_counts()	Tiện lợi và đơn giản nhất trong pandas
Tính toán số học	Vòng lặp Python thuần	Rất chậm (ví dụ: mean_distance chậm hơn 100 lần).
	NumPy Vectorized (phép toán mảng)	Nhanh hơn nhiều so với Python thuần
	Numba JIT (nopython=True)	Nhanh hơn cả phiên bản NumPy được vectorized
Sắp xếp	quicksort (mặc định)	Nhanh hơn về mặt trung bình, không ổn định

	mergesort	Ổn định (bảo tồn vị trí tương đối của các phần tử bằng nhau), hiệu suất O(n log n) được đảm bảo.
Tạo funcs	numpy.frompyfunc	Trả về mảng đối tượng Python; bất tiện
	numpy.vectorize	Cho phép chỉ định kiểu đầu ra, nhưng rất chậm do gọi hàm Python cho mỗi phần tử.
	numba.vectorize	Tạo ra các ufuncs NumPy được biên dịch, hoạt động nhanh như các ufuncs tích hợp sẵn.

3. Bài học kinh nghiệm khi làm việc với dữ liệu thực tế

Khi làm việc với các bộ dữ liệu thực tế, các nguồn tài liệu đã làm nổi bật một số thách thức và bài học quan trọng.

Dữ liệu thực tế hiếm khi sạch, thường chứa các giá trị thiếu hoặc không xác định (ví dụ: trường múi giờ tz trong Bitly, cột nhà sản xuất trong USDA). Việc kiểm tra sự tồn tại của các trường và xử lý các giá trị thiếu/không xác định là bước bắt buộc.

Khi dữ liệu được phân tán trên nhiều bảng (như MovieLens), việc hợp nhất chúng thành một bảng duy nhất (sử dụng pd.merge) là cần thiết để đơn giản hóa các tác vụ phân tích phức tạp.

Các trường dữ liệu dạng chuỗi đòi hỏi các bước lập bản đồ (mapping) để chuẩn hóa các mục nhập không nhất quán (ví dụ: nhiều biến thể của nghề nghiệp FEC) thành các danh mục thống nhất.

Để đạt được hiệu suất tối ưu, cần chuyển đổi logic dựa trên vòng lặp Python sang các phép toán mảng vectorized của NumPy và tận dụng tính năng Broadcasting, tuân thủ chặt chẽ Quy tắc Broadcasting.

Thực hiện Kiểm tra Tính hợp lệ (Sanity Checks): Sau khi thực hiện các phép toán nhóm phức tạp (ví dụ: tính tỷ lệ), việc thực hiện kiểm tra tính hợp lệ (chẳng hạn như xác minh rằng tổng tỷ lệ trong mỗi nhóm bằng 1.0) là rất quan trọng để đảm bảo độ chính xác của kết quả.

Sử dụng Công cụ Cấp cao: Việc sử dụng các công cụ cấp cao của pandas như pivot_table, groupby, và value_counts cung cấp cách tiếp cận dễ đọc, ngắn gọn và hiệu quả hơn so với việc cố gắng thực hiện cùng một phân tích chỉ bằng thư viện chuẩn Python.

4. Hướng phát triển và ứng dụng trong tương lai

Các kỹ năng phân tích dữ liệu được trình bày trong các nguồn tài liệu sẽ tiếp tục có giá trị trong tương lai, và có nhiều hướng để phát triển và ứng dụng.

Các bộ dữ liệu như MovieLens (chứa ratings, metadata phim và thông tin nhân khẩu học người dùng) là nền tảng để phát triển các hệ thống gợi ý dựa trên các thuật toán học máy.

Tiếp tục phân tích các xu hướng chưa được khám phá trong dữ liệu (ví dụ: phân tích sự gia tăng các cách viết thay thế, hoặc tìm ra các tên chuyển đổi từ giới tính này sang giới tính khác).

Phân tích quyên góp có thể được tinh chỉnh bằng cách tổng hợp đóng góp theo tên nhà tài trợ và mã zip để điều chỉnh cho những người đóng góp nhiều khoản nhỏ so với một khoản lớn.

Đối với các thuật toán yêu cầu hiệu suất cao không thể được vector hóa hoàn toàn bằng NumPy, Numba có thể được sử dụng để dịch mã Python thành mã máy được biên dịch, giúp tạo ra các hàm nhanh hơn đáng kể. Numba có thể tạo các ufuncs tùy chỉnh được biên dịch, hoạt động nhanh như các ufuncs tích hợp sẵn của NumPy.

Với các bộ dữ liệu lớn không thể nằm vừa trong RAM (hàng trăm gigabyte hoặc terabyte), Memory-Mapped Files (memmap) trong NumPy cho phép tương tác với dữ liệu nhị phân trên đĩa như thể nó là một mảng trong bộ nhớ, cho phép đọc và ghi các phân đoạn nhỏ.

Sử dụng các định dạng lưu trữ hiệu quả và nén được như HDF5 (thông qua PyTables hoặc h5py) là một giải pháp để lưu trữ các mảng dữ liệu lớn.

Trong các ứng dụng hiệu suất cao, việc hiểu và tận dụng bố cục bộ nhớ liên tục (C-order hoặc FORTRAN-order) của mảng có thể ảnh hưởng đáng kể đến tốc độ tính toán do cách thức CPU cache hoạt động.